Basil Manual

PIC18F27Q10 · basil 0.1.29

2026-10-26

Basil Manual

Contents

About this manual ... 4

1. Compiler overview ... 5
- Supported device ... 5
- Running the compiler ... 5
- Debugging ... 6
- Program layout ... 8
- Identifiers, labels, comments, line continuation ... 8
- Numeric representations and literals ... 8
- Variables and types ... 9
- Aliases and register bits ... 11
- Constants and Symbol ... 12
- Arrays, strings, structures ... 12
- Float ... 13
- Operators and expression typing ... 14
- Procedures (Sub, Function, Proc) ... 15
- Interrupts, events and context saving ... 15
- Modules and Include ... 20
- Device, Clock, Config ... 20
- Preprocessor ... 21

2. Language commands ... 22
- Abs ... 22
- AddressOf (and `@`) ... 22
- Asm ... End Asm ... 23
- Bound ... 24
- Break ... 24
- Clear ... 25
- Continue ... 25
- Dec ... 25
- DelayMS / DelayUS ... 26
- Disable / Enable ... 27
- Do ... Loop ... 27
- Exit / ExitProc ... 28
- For ... Next ... Step ... 28
- GoTo ... 29
- High / Low / Toggle / Input / Output ... 29
- If ... Then ... ElseIf ... Else ... EndIf ... 30
- Inc ... 30
- Min / Max ... 31
- Repeat ... Until ... 32
- Save ... Restore ... 32
- Select ... Case ... EndSelect ... 33
- Terminate ... 34
- While ... Wend ... 34

3. Library modules ... 35
- gpio (`Include "gpio"`) ... 35
- delay (`Include "delay"`) ... 36
- intosc (`Include "intosc"`, internal oscillator set-up) ... 37

system (`Include "system"` , reset cause and clock check) ... 38
timer (`Include "timer"` , Timer0, Timer1, Timer2) ... 40
serial (`Include "serial"` , EUSART1, 8N1, TX = RC6, RX = RC7) ... 48
serial2 (`Include "serial2"` , EUSART2, TX = RB6, RX = RB7) ... 51
adc (`Include "adc"` , ADCC, 10-bit, right-justified) ... 52
eeprom (`Include "eeprom"` , data EEPROM, 1024 bytes) ... 54
pwm (`Include "pwm"` , CCP1, CCP2, PWM3, PWM4 on Timer2) ... 56
i2c (`Include "i2c"` , MSSP1 host) ... 59
spi (`Include "spi"` , MSSP1 host) ... 62
lcd (`Include "lcd"` , HD44780 character LCD, 4-bit) ... 64
oled (`Include "oled"` , SSD1309, 128 x 64, I2C or 4-wire SPI) ... 68
aht30 (`Include "aht30"` , AHT30 temperature and humidity sensor, I2C) ... 73
floatlib (automatic) ... 73

4. Diagnostics ... 74

Appendix A: Alternative spellings and unsupported forms ... 75
Accepted alternative spellings ... 75
Not available (or different) ... 75

Appendix B: Test checklist ... 77

About this manual

Revision for **basil 0.1.21**, target device **PIC18F27Q10**.

This manual describes exactly what the compiler accepts and generates today. It gives an overview of the language, then one entry per command (Syntax, Overview, Parameters, Example, Notes), then one entry per library routine. Anything not listed here is not supported; Appendix A lists the alternative spellings that are accepted and the common BASIC features that are missing or different, and Appendix B is a checklist to work through when testing.

Status of the code: every feature below passes the Microchip MDB simulator suite and the host test suite. **Nothing has been run on a real chip yet** (decision D6). Where a feature depends on pin behaviour the simulator cannot model (serial bytes on the wire, PWM waveforms, external timer edges, I2C/SPI traffic, LCD/OLED panels), that is stated, and those are the items to test on hardware first.

Related documents: `docs/LANGUAGE.md` (formal language rules), `docs/GRAMMAR.md`, `docs/LIBRARY.md`, `docs/DIAGNOSTICS.md`, `docs/CLOCK.md`, `docs/ABI.md`, `QUICKSTART.md`.

1. Compiler overview

Supported device

Device	Core	Program memory	RAM	Status
PIC18F27Q10	PIC18 (8-bit), legacy instruction set (`XINST = 0`)	128 KB (65,536 words)	3,359 usable GPR bytes (as reported by the build summary)	**Supported.** Device file `devices/18F27Q10.toml`, generated from pack `PIC18F-Q_DFP 1.31.492`. All library modules
PIC16F17126 (14/16-pin)	PIC16 enhanced mid-range (`pic16e`)	16K words (28 KB)	2,048 bytes	**Described, not compilable yet.** Device file `devices/16F17126.toml` (pack `PIC16F1xxxx_DFP 1.32.471`, data sheet DS40002343F). There is no `pic16e` code generator, so `Device = 16F17126` stops with `E-DEVICE`. Library modules `gpio`, `intosc`, `system`, `timer`, `serial` and `adc` are prepared for it; the others are not
PIC16F17146 (20-pin)	PIC16 enhanced mid-range (`pic16e`)	16K words (28 KB)	2,048 bytes	As the PIC16F17126 (`devices/16F17146.toml`)

`Device = 18F27Q10` (or `PIC18F27Q10`) is the only value that compiles today. A device is added only after it has its own description file and passes its own tests, and **adding one means more than the description file**:

- a new core family needs its own code generator (PLAN §9.3);
- every library module must be reviewed for the new part: the modules were written for the PIC18F27/47Q10 and each one names registers, interrupt flags, pins and channel numbers that can differ or not exist. The state of each module per device is in the table in `docs/LIBRARY.md`, "Device support: work still to do". A module that is not ported must stop with `#error`, not use the Q10 registers;
- this section and that table must be updated, and the new part listed here.

Every register and register bit of the device is available by its pack name (`PORTB`, `LATA`, `T0CON0`, `PIE0.5`, `SSP1CON1.SSPEN` ...), case-insensitively. A program or library module can ask whether the chosen device has a register or bit field with `#if _has(REG)` / `#if _has(REG.FIELD)` (Preprocessor, below), which is how one module serves several layouts.

Running the compiler

```
basil [--check] [-S] [--report] [-O0] [-g] [-o OUT.hex] [-D NAME] [-L LIBDIR]
       [--devices DIR] [--pic-as PATH] [--version] <program.bas>
```

Option	Meaning
(none)	Compile, assemble and link. Writes `prog.hex`, `prog.S` (assembly), `prog.map`.

Option	Meaning
`--check`	Front end only (syntax, types, declarations). No output files. Fast.
`-S`	Stop after writing the assembly (`prog.S`).
`--report`	Also print words per routine and every RAM object.
`--keep-vars`	Keep stores to variables that nothing reads (the optimiser normally drops them), so a test can inspect them in RAM. A `Volatile` variable is always kept.
`-g`	Debug build: adds line, variable and routine information to `prog.elf` for a debugger (see Debugging). The hex file is identical to a build without it.
`-O0`	No optimisation (the test suite builds everything both ways and compares). `-O1` , `-O2` , `-Os` are accepted and mean "optimise" (the default).
`-o FILE`	Output hex path.
`-D NAME`	Define a preprocessor name (visible in every file).
`-L DIR`	Add a library folder (repeatable).
`--devices DIR`	Folder holding the device descriptions.
`--pic-as PATH`	The `pic-as` assembler (otherwise `$BASIL_PICAS` , then `/opt/microchip/xc8/*/pic-as/bin/pic-as`).
`--version`	Print the version.

`lib/` and `devices/` are found by looking in the source file's folder and then each parent folder (only the nearest of each is used), so keep programs inside the repository (for example `my_src/`) or pass `-L` and `--devices` .

After a successful build the compiler prints the program size and a bar of flash and RAM used against the device (colour only on a terminal, and not when `NO_COLOR` is set):

```
Program: 28 words (56 bytes)
basil: blink.bas -> blink.hex (28 words, 56 bytes)
```

Errors print as `file:line:col: error[CODE]: message` , with the source line and a caret. Several errors are reported per run. Exit status is non-zero if any error was reported.

Debugging

`basil -g prog.bas` builds as usual and also makes `prog.elf` describe the program to a debugger in terms of the BASIC source:

- **Lines.** Every statement is preceded by a `#line` directive in `prog.S` , so pic-as writes a DWARF line table that points at the `.bas` files (library modules included), not at `prog.S` . Breakpoints and stepping work on BASIC lines. Code the compiler adds itself (start-up, runtime helpers) is shown under the nearest preceding statement.
- **Variables and routines.** After pic-as has run, basil appends one DWARF unit to the ELF: each global, and each routine's parameters and locals, with their types and fixed RAM addresses (`Byte` , `ShortInt` , `Word` , `Integer` , `LongWord` , `LongInt` , `Float` , `Char` , `Boolean` , strings and arrays; structures appear as their bytes). Interrupt handlers and routines that were not inlined appear as functions, so the call stack shows names and parameter values. The locals of a routine the

compiler inlined into its one caller are not listed. Several routines may share RAM addresses, so only the variables of the routine being stopped in are meaningful.
- **Symbols.** Each variable is also a real ELF symbol (`BLINK_TICKS`, the assembly name), and a global is also known by its BASIC name (`ticks`), so command-line debuggers can find it.

`-g` changes nothing in the generated code: the hex file is byte-identical with and without it (checked over the whole `tests/exec` suite). A build without `-g` carries none of this information. `prog.hex` and `prog.elf` are written next to the source (or beside `-o`).

Debugging with the MPLAB X simulator (`mdb`)

Put these lines in `cmds.txt`:

```
device PIC18F27Q10
hwtool sim
program prog.elf
break prog.bas:37
run
wait 5000
where
print ticks
step
quit
```

then run `basil -g prog.bas` and `/opt/microchip/mplabx/v*/mplab_platform/bin/mdb.sh cmds.txt`. `break file:line` takes a BASIC line, `where` shows the routine with its parameters, `print NAME` prints a variable and `step` moves to the next BASIC line. A library routine is found by its own file (for example `serial.bas`).

Debugging in VS Code

The VS Code extension (`vscode-extension/`) runs the debug build and starts Microchip's debugger on the result. It needs the *MPLAB Extension Pack* and the device pack of the target. Hardware (PICkit, ICD, Snap) is only needed for a real chip; the simulator needs none.

Command (Ctrl+Shift+P, "PIC BASIC: ...")	Key	What it does
New Project		Creates a folder with the `.bas`, `picbasic.json`, `.vscode/launch.json`, `tasks.json` and `dist/`.
Pick Target Device		Changes the `Device` line, `picbasic.json` and `launch.json`.
Select Programmer/Debugger		Simulator or a connected PICkit / ICD / Snap, stored per project.
Build	F7	`basil`, then copies `.hex`, `.elf`, `.map` into `dist/`.
Build Debug	Ctrl+F7	`basil -g`, then the same copy.
Program Device	F8	Builds and programs `dist/<name>.hex` with the selected hardware tool.
Show Build Output		Opens the output channel.

Start debugging with F5 (configuration `PIC BASIC: Debug`; F9 breakpoint, F10 / F11 / Shift+F11 step). Run Build Debug first, or the debugger will not find `dist/<name>.elf`. The extension's settings are listed in `vscode-extension/README.md`.

Program layout

A source file is a **program** or a **module**.

```
Program Name                        ' name is optional
    Device = 18F27Q10               ' required, before any Include or declaration
    Clock  = 16                     ' MHz; required for anything using delays or timing
    Config FEXTOSC = OFF, RSTOSC = HFINTOSC_64MHZ, WDTE = OFF, LVP = ON
    Include "module"                ' zero or more
    ' constants, types, variables, aliases
    ' subroutines, functions, interrupts, events
    ' statements - the main program
```

- There is no `Main`. The top-level statements run in order. Statements of included modules (their initialisation) run first, in include order.
- A program that runs off its end loops forever (a self-branch); it never runs into empty flash.
- **Order is enforced**: Device/Clock/Config, then `Include`, then constants/types/variables/aliases, then routines, then statements. After the first executable statement, `Include`, `Dim` and routine declarations are an error (`E-ORDER`).
- Identifiers must be declared before use. There is no forward reference.
- The program and all modules are compiled together. A module included by several files is loaded once.

A `Proc ... EndProc` flat/procedural layout is accepted (see *Procedures*).

Identifiers, labels, comments, line continuation

- **Identifiers**: letter or `_`, then letters, digits and `_`. Not case sensitive (`Label`, `LABEL` and `label` are the same). A variable cannot have the name of a device register (`trisa` clashes with `TRISA`; `W-REGNAME`).
- **Reserved words** cannot be names (`E-RESERVED`). Notable ones that catch people out: `Inc`, `Dec`, `High`, `Low`, `Input`, `Output`, `Toggle`, `Clock`, `Device`, `Type`, `Result`, `Step`, `Next`, `Event`, `Loop`, `Do`, `Symbol`, `Proc`, `Declare`. The full list is in `src/lexer.rs` (`KEYWORDS`).
- **Labels**: `name:` as the first thing on a line with nothing after the colon. Labels are local to the routine (or to the top-level code) and must be unique there. Jump with `GoTo name`.
- **Comments**: `'` or `//` to end of line; `(* ... *)` or `{ ... }` for blocks (these nest inside each other, but not themselves with mixed closers).
- **Several statements on one line**: separate with `:`.
- **Line continuation**: end a line with a space and `_`.

Numeric representations and literals

Form	Example	Notes
Decimal	`100`	
Hexadecimal	`$FF`, `0xFF`	`0x` or `0X`; digits in either case
Binary	`%1010`, `0b1010`	`0b` or `0B`
Floating point	`3.14`, `1.0e3`, `1.5E-2`	

Form	Example	Notes
Character	`"A"`	a Char, or a one-character String, by context
Character code	`#65`, `#13 + #10`	Char with the given code; can be concatenated
String	`"Hello"`	up to 255 characters
Booleans	`True`, `False`	
`Null`	`Null`	string terminator / empty string

Variables and types

```
Dim Name As Type
Dim Name1, Name2 As Type
Dim Name As Type = constant          ' initial value
Dim Name = constant, Name2 = 5 As Type
```

Type	Bits	Range / meaning	Alternative spelling
`Boolean`	1	`True` / `False`. Cannot hold a number	
`Bit`	1	0 / 1	`Bit`
`Byte`	8	0 to 255	`Byte`
`ShortInt`	8	-128 to 127	`SByte`
`Word`	16	0 to 65,535	`Word`
`Integer`	16	-32,768 to 32,767	`SWord`
`LongWord`	32	0 to 4,294,967,295	`Dword`
`LongInt`	32	-2,147,483,648 to 2,147,483,647	`Long`
`Float`	32	IEEE-754 binary32, about +/-3.4e38, 6 to 7 digits	`Float`
`Char`	8	one character	
`String(n)`	n+1 bytes	up to n characters plus a terminating `Null`	
`Structure`		see *Structures and unions*	

Note on `Long`: here `Long` is the signed 32-bit type (`LongInt`), not a 24-bit unsigned `Long`. There is no 24-bit type; 24 bits is only used inside the compiler. `SDword` and `Double` are not accepted. A `Pin` type is not accepted.

- All variables are zeroed at start-up unless they have an initialiser. A routine's local variables are zeroed **at every entry**, and an initialiser is applied at every entry too (use `Static` to keep a local between calls).
- Declarations are `Private` by default; `Public` exports from a module.
- Name lookup order: local, then the current file, then the `Public` names of included modules. Qualify with `Module.Name` to resolve a clash.
- Global variables must fit the device RAM (`E-RAM`). The hardware return stack is checked at compile time (31 levels; `E-STACK`).

Type conversion

`Byte(x)`, `Word(x)`, `Integer(x)`, `LongWord(x)`, `LongInt(x)`, `ShortInt(x)`, `Float(x)`, `Boolean(x)`, `Bit(x)`, `Char(x)`, and the alternative spellings `SByte(x)`, `SWord(x)`, `Dword(x)`, `Long(x)`. A constant that does not fit the target type is an error (`E-CONST-RANGE`). Conversion to `String` is not supported. Float to integer truncates toward zero.

`Static` locals

```
Static Dim Name As Type [= constant]
Static Dim Name(n) As Type [= (v1, v2, ...)]
```

Overview. An ordinary local is created afresh each time its routine runs: it is zero (or its initial value) at every entry, and its RAM is shared with other routines. A `Static` local instead keeps its value from one call to the next, like a `static` variable in C. It is still visible only inside the routine that declares it.

What the compiler does. A `Static` local gets its own RAM address that no other routine uses. It is cleared once at start-up, and an initial value (including a whole array or string initialiser) is applied once before the main program starts, not at each call. The cost is that the RAM is permanently reserved, whereas an ordinary local costs nothing when its routine is not running.

Rules.

- Only inside a `Sub`, `Function`, `Interrupt` or `Event`. At module level every variable already lasts for the whole run, so `Static Dim` there is error `E-DIM`.
- Applies to a variable, not an alias (`E-DIM`). It can be combined with `Volatile`, in either order.
- It may be used in an interrupt handler (the value then persists between interrupts). `Save ... Restore` does not save or restore it, because it is not part of the routine's overlaid frame.
- There is one copy of it, shared by every caller. If a routine with a `Static` local is called from both the main program and an interrupt handler, they update the same variable, exactly as with a module-level variable; mark it `Volatile` as well if either side polls it.

Example.

```
Function NextId() As Byte
    Static Dim id As Byte = 100     ' set to 100 once, at start-up
    Inc(id)
    Result = id
End Function

Dim a, b As Byte
a = NextId()    ' 101
b = NextId()    ' 102
```

`Volatile` variables

```
Volatile Dim Name As Type
Private Volatile Dim Name As Type
Static Volatile Dim Name As Type      ' inside a routine
```

Overview. `Volatile` tells the compiler that the variable can change, or be observed, by something outside the code it is compiling: an interrupt handler, an `Asm` block, or the hardware (for example a variable that mirrors a register). Every read and every write written in the source is carried out, in the order written.

What changes. Normally the optimiser may keep a value in `WREG` or a scratch register and reuse it, merge two reads into one, replace a variable by a known constant, or delete a store to a variable that is never read. For a `Volatile` variable it does none of these:

Source	Ordinary variable	`Volatile` variable
`x = flag : y = flag`	`flag` may be read once	read twice
`flag = 1 : flag = 2`	first store may be dropped	both stores happen
write to a variable nothing ever reads	store removed	store kept
`Do : If flag = 0 Then ... : Loop`	may reuse a loaded value	reloaded on every pass

Rules and cost. Allowed on any scalar, array, string or structure `Dim` at any level; not on an alias (`E-DIM`). A volatile access can be a word or two larger than the same access to an ordinary variable, so use it only where it is needed. `Volatile` does not make a multi-byte access atomic: a `Word` or `LongWord` shared with an interrupt handler can still change between its bytes; read it between `Disable(Handler)` and `Enable(Handler)`, or keep the shared value to one `Byte`.

Example. A flag set by a handler and polled by the main program:

```
Volatile Dim tick As Byte

Interrupt Timer()
    tick = 1
    ' clear the flag of your interrupt source here
End Interrupt

' set up the source and its PIEx enable bit first
Enable(Timer)
Do
    If tick = 1 Then          ' reloaded on every pass of the loop
        tick = 0
        ' ... handle the event ...
    End If
Loop
```

Aliases and register bits

An alias creates a second name for existing storage; it uses no RAM.

```
Dim LED As PORTB.0              ' a register bit
Dim Flags As Byte
Dim Ready As Flags.0            ' a bit of a variable
Dim LowNibble As Flags.Bits(4)  ' see the modifier table
Symbol Button = PORTA.3         ' alternative spelling, same effect
```

Modifier	Gives
`.n` or `.Bits(n)`	bit n
`.Booleans(n)`	bit n as a Boolean
`.Byte0` … `.Byte3`, `.Bytes(n)`	byte n of a multi-byte value
`.Word0`, `.Word1`, `.Words(n)`	word n
`.AsLongWord` and similar	a larger view of smaller storage (promotion), for example `Dim L As Arr(0).AsLongWord`

Aliases may chain. A bit alias is a `Bit`; it cannot be used directly as a condition (`E-COND`): write `If LED = 1 Then`. Any device register or register bit can be read and written by name without declaring it: `PORTB`, `TRISB`, `ANSELB = 0`, `PIE0.5 = 1`, `SSP1CON1.SSPEN = 1`.

Constants and Symbol

```
Const Name = expression               ' type inferred
Const Name As Type = expression
Symbol cLimit = 100                   ' the = is optional
Symbol LED = PORTB.0                  ' becomes an alias, chosen by what the value is
```

A constant without a type takes its type from the expression: `5 / 1024` is integer arithmetic and is 0; write `5.0 / 1024` for floating point. Constants can be used in array sizes, `Case` values and anywhere a constant expression is required. `_Clock` and `_Device` are predefined constants (the `Clock` value and the device name).

Arrays, strings, structures

Arrays

```
Dim a(5) As Byte                       ' FIVE elements: a(0) .. a(4)
Dim t(3) As Byte = (1, 2, 3)           ' initialiser: exactly 3 values
Const Tbl(4) As Byte = (10, 20, 30, 40)  ' lives in program memory
```

- The number in brackets is the **element count**. `Bound(a)` is the highest index (4 here), so `For i = 0 To Bound(a)` visits every element.
- Element types: the integer types, `Char`, `Boolean`, `Bit`, `Float`. One dimension only.
- A constant index outside the array is a compile error (`E-BOUNDS`). A run-time index is not checked.
- `Const` arrays are read-only (reading by index works; assigning a `Const` array to a same-sized RAM array copies it).
- `Bit` and `Boolean` arrays that are only indexed are packed eight to a byte. Writing one through a run-time index is a read-modify-write: do not write other elements of the same array from an interrupt (the compiler turns interrupts off around the write).
- Arrays are passed to routines as `ByRef` (written `p() As Byte`) or `ByRefConst`.
- `Clear(a)` zeroes an array; `a = b` copies same-sized arrays.
- Not supported: multi-dimensional arrays, arrays of strings, `Const` arrays of Float, functions that return an array.

Strings

```
Dim s As String(20)                    ' 21 bytes
Dim t As String(10) = "hi"
s = "abc"                              ' from a literal, another string, a Char
s = s + "def"                          ' in-place append
s = "x" + t + "y"                      ' + chain (the target may only be the first operand)
If s = "abc" Then ...                  ' = and <> only
c = s(1)                               ' element, zero-based, a Char
Clear(s)
```

Assignments are cut off at the declared size and always terminated with `Null`. There is no run-time bounds check on `s(i)`. String literals live in program memory and are read with `TBLRD`. A `String` parameter is `ByRef` (size must match the callee's `String(n)` exactly) or `ByRefConst` (any size, also accepts a literal).

Not supported: `Len`, `Left`, `Mid`, `Right`, `Str`, `Val`, `Print`, comparison of a concatenation, string-returning functions, arrays of strings. Output of text is done by the library routines (`SerialWrite`, `LcdWrite`, `OledWrite`).

Structures and unions

```
Structure TStatus
    Val       As Byte
    Enabled   As Val.0           ' alias field: shares storage with Val (a union)
    Connected As Val.1
End Structure

Structure TPoint
    X As Word
    Y As Word
End Structure

Dim p As TPoint
Dim pts(4) As TPoint             ' arrays of structures; structures may nest
p.X = 100
pts(2).Y = 7
Clear(p)
```

Fields are reached with `.`. `Clear(x)` zeroes a structure. Structure initialisers are not supported.

Float

`Float` is IEEE-754 binary32 (sign, 8-bit exponent, 23-bit fraction). It is implemented in `lib/floatlib.bas`, which the compiler adds automatically to any program that mentions `Float`; only the routines used are linked.

- Supported: variables, constants, arrays, parameters, results; `+ - * /`; unary minus; `Abs`; the six comparisons; mixed integer and Float expressions (the integer is converted); `Float(x)` and the integer conversions `Byte(f)`, `Integer(f)`, ... (truncating).
- Rounding is to nearest even. Infinities and NaN propagate. Denormals are flushed to zero.

- Not supported: `Sqr`, `Sin`, `Cos`, `Tan`, `Log`, `Exp`, `Pow`, `fRound` and the other math functions; `Print`/`Str` of a Float; a Float as a `For` variable, in `Select`, or with `Inc`/`Dec`; `=` and `<>` between floats are available but avoid them as for any float.
- Cost: about 700 words for `+`, 800 for `*`, 470 for `/`, plus about 250 shared (roughly 2,500 words for a program using all three).
- In an interrupt handler, float arithmetic is an error (`E-ISR-CALL`): do it in a routine and call that inside `Save(...) ... Restore`. Copying a Float, negation and `Abs` are allowed. A 4-byte Float is not read atomically.

Operators and expression typing

Highest precedence first:

Level	Operators
1	`@` (address), unary `-`, unary `+`, `Not`
2	`*`, `/`, `Mod`
3	`+`, `-`
4	`<<`, `>>`
5	`=`, `<>`, `<`, `>`, `<=`, `>=`
6	`And`
7	`Or`, `Xor`

- `And`, `Or`, `Xor`, `Not` are **logical** on Booleans and **bitwise** on integers. Mixing a Boolean and a number is an error (`E-MIX`). Conditions short-circuit left to right.
- There is no `\`, `^`, `&`, `|`, `~`, `<<=` or `Dig`/`?`, `@` bit reverse, `Dcd`/`Ncd`.
- **Evaluation width.** An integer expression is evaluated at the width of its widest operand or its target, whichever is larger, and no wider. `B3 = B1 + B2` is an 8-bit add that wraps; `W = B1 * B2` is a full 16-bit product (hardware 8 x 8 multiply). A comparison is evaluated at the wider of its two sides.
- **Mixing signedness.** Signed and unsigned operands of the same width cannot be combined without a conversion (`E-SIGN`); a narrower unsigned operand is widened safely; a narrower signed operand against a wider unsigned one is rejected. Unary minus on an unsigned variable widens to the next signed type.
- **Narrowing.** A constant that does not fit is an error; a non-constant narrowing is the warning `W-NARROW` (error under `#option Strict = true`).
- **Division and `Mod`** truncate toward zero; `Mod` takes the dividend's sign. Divide by zero at run time gives an all-ones quotient and the dividend as remainder; in a constant expression it is an error. A division by a power-of-two constant becomes a shift. A constant multiplier becomes shifts or `MULLW` steps.
- **Shifts.** The count must be a constant or a `Byte`. Shifting by the width or more gives 0 (or the sign for a signed right shift). `>>` is arithmetic for signed types.
- Functions may be used in expressions; subroutines may not. Parentheses are needed when calling with arguments.

Procedures (Sub, Function, Proc)

```
[Public|Private] [Inline|NoInline] Sub Name([params])
    ' declarations and statements
End Sub

[Public|Private] [Inline|NoInline] Function Name([params]) As Type
    ' assign to Result (or to the function name)
End Function

param := [ByVal | ByRef | ByRefConst] Name [()] As Type [= constant]
```

Equivalent alternative forms are accepted:

```
Proc Name(p As Byte)                ' = Sub
    ...
    ExitProc                        ' = Exit
EndProc

Proc Double(pValue As Byte), Word   ' = Function ... As Word, value in Result
    Result = pValue * 2
EndProc
```

- Parameters are **by value** by default for scalars; arrays, `String` and structures are `ByRef` (or `ByRefConst`). A `ByRef` argument must be a variable, not an expression. A `ByRef` parameter can be a Byte/Word/etc. scalar, an array, a String or a structure.
- A default value is allowed only on a trailing `ByVal` parameter, and must be a constant.
- A function returns a scalar or a structure (not an array, not a String). Calling a function as a statement discards its result.
- **No recursion** (`E-RECURSION`). Because the call graph is acyclic, the compiler gives every routine static storage and **overlays** the storage of routines never live at the same time, so RAM use is the deepest call chain, not the sum. Declaring working variables locally saves RAM. `Reentrant` is parsed but rejected by the back end.
- A routine called once is inlined by default; `Inline` forces and `NoInline` forbids it.
- Overloading and compound subroutines are not supported.
- Routines are called as `Name(args)`, or `Module.Name(args)`.

Interrupts, events and context saving

Interrupts

```
Interrupt Name([priority])
    ' handler body
End Interrupt
```

An interrupt handler is a routine the hardware calls when an enabled source raises its flag. It has no parameters, no result, and is never called by name.

- With one handler, write no priority: the compiler picks the vector. With two handlers, the low one is `Interrupt Name(ipLow)` (`ipLow = 1`) and the high one `Interrupt Name(ipHigh)` (`ipHigh = 2`). More handlers than vectors may be *declared*, but only one per priority can be *enabled*.
- The 18F27Q10 has fixed vectors. A handler is not attached to its vector until `Enable` commits it.
- Library modules never declare an `Interrupt`; they set up peripherals and read flags, and your program writes its own handler.
- A handler declares its own locals, which have their own storage and are never overlaid with main-line routines.
- **Clear the source's flag inside the handler.** Hardware never clears it, so a handler that forgets re-enters forever.
- The handler runs between any two instructions of the main program. Keep it short; a wide variable (Word, LongWord, Float) shared with the main program is **not** read or written atomically, so copy it with the interrupt disabled or use a Byte flag. Cycle-counted delays (`DelayMS`, `DelayUS`) run longer when interrupted.
- `Exit` leaves the handler early; its end restores the context and returns.

Enabling and disabling

```
Enable(Name)        ' commit the handler to its vector and set the global enable
Disable(Name)       ' clear the global enable bit(s)
```

`Enable` and `Disable` touch only the **global** bits (`GIE/GIEH`, `PEIE/GIEL`). Turning on one source is the program's job, in this order:

1. configure the peripheral;
2. clear its flag (`PIRx`);
3. set its enable bit (`PIEx`);
4. with two priorities, set its priority bit (`IPRx`; they reset to *high*);
5. `Enable(Name)`.

`Disable` does not remove the handler from its vector; `Enable` can turn it back on. Enabling two handlers of the same priority is an error (`E-ENABLE`). Clearing the global bit inside a handler is undone by the return from it.

Context saving

The compiler saves what a handler needs, but a handler is not a normal routine.

- **Hardware shadow registers.** `STATUS`, `WREG` and `BSR` are saved in the fast register stack for a lone handler. With two priorities the high one uses the shadow registers, and the low one saves and restores the same three in software (a high interrupt would overwrite the shadow copies). `#option ISR_SHADOW = false` forces software saving for both.
- **Nothing else is saved.** Any statement that needs a library routine, a helper, or a routine's frame (a call, or Float arithmetic) would use storage the interrupted main program may be using. The compiler refuses it with `E-ISR-CALL` unless you protect it with `Save`.

```
Save(item {, item})
    ' statements
Restore
```

Each `item` is a variable, a routine, or the constant `0`. `0` saves the compiler's working registers (including `FSR0` and `FSR1`). A routine item saves that routine's storage **and that of everything it calls**. Inside a handler the compiler also adds the storage of every routine the block calls, so the usual form is `Save(0, Routine)` around a call to `Routine`.

```
Dim total As Word
Function Cube(ByVal v As Byte) As LongWord
    Result = v * v * v
End Function

Interrupt MyInt()
    Dim big As LongWord
    Save(0, Cube)              ' working registers + Cube's parameter, temporaries and
result
        big = Cube(10)
    Restore
End Interrupt
```

Without the `Save`, `Cube` would overwrite RAM the main program's own routines share with it, and the interrupted routine would resume with damaged data. Decide what to save by reading the datasheet, by looking at the routine you call, and by reading the generated `.S` file for registers it touches.

`Exit`, `Break` and `Continue` that leave a `Save` block restore first. `GoTo` and labels inside it are not supported. Float arithmetic, comparisons and conversions in a handler are `E-ISR-CALL` directly: do them in a routine and call it inside `Save`. Copying a Float, `-f` and `Abs(f)` are plain bit operations and are allowed.

Events

An **event** lets one piece of code (a library module, say) run a handler that *another* piece of code supplies, without either knowing about the other. It is the safe way to "plug code into" an interrupt: the module owns the interrupt and its hardware, and the program supplies what happens.

```
Type TEvent = Event()           ' 1. a type of handler
Event Beep()                    ' 2. a handler
    Toggle(PORTB.0)
End Event
Dim OnTick As TEvent            ' 3. a variable that can hold one
OnTick = Beep                   ' 4. point it at the handler
OnTick()                        ' 5. call through the variable
```

```
Type TName = Event([ByVal param As Type {, ByVal param As Type}])
[Public|Private] Event Name([ByVal param As Type {, ByVal param As Type}])
    ' declarations and statements
End Event
```

- **An event cannot be called by its name.** Only through a variable of an event type (`Beep()` is `E-CALL`). The variable is what you hand to a module as a parameter.
- **Parameters.** An event type and its handlers can take `ByVal` parameters of an integer type, `Boolean` or `Char` (nothing else: no `ByRef`, arrays, strings, structures or default values, `E-PARAM`). A handler fits a variable only if its parameter types are exactly those of the type (`Type TRx = Event(ByVal c As Byte)` takes `Event OnByte(ByVal c As Byte)` but not `Event OnByte(ByVal c As Word)`; the assignment is `E-TYPE`). The parameter names may differ. A call gives the right number of arguments (`E-ARGS`), each converted as for a routine.
- An event has no result. To get data back, use a variable both sides can see.
- A handler name is assigned to a variable of an event type (`OnTick = Beep`). A variable that has never been assigned does nothing when called, so a module can call its event without testing it. Assign again to change the handler at run time.
- **Comparing.** `=` and `<>` compare two event variables of one type, or a variable and a handler name: `If OnTick = Beep Then ...`. Nothing else compares (`<`, `>` and so on are `E-TYPE`).
- **Arrays.** `Dim Table(4) As TEvent` makes five event variables, called through an index: `Table(i)()` or, with arguments, `Table(i)(3)`. An array of events starts empty: assign its elements one by one (`Table(0) = Beep`); an initialiser list is refused (`E-DIM`). Unassigned elements do nothing.
- A variable of an event type holds **one byte**: 0 for none, otherwise the number of a handler. It is not an address.
- Event variables can be copied (`B = A`), passed `ByVal` to a routine, and be module-level or local variables.
- Only handlers that are named in an assignment or comparison are compiled. An event nobody assigns costs nothing.
- `Event` is `Private` by default; mark it `Public` to use it from another module. The type, the variable and the parameter use the same `Public`/`Private` rules as any other name.
- `Exit` leaves the handler early.

How a call works. A call through a variable is a call to one shared dispatcher routine (one per parameter list), which compares the byte and calls the matching handler, passing the arguments on. Handlers are ordinary routines to the compiler: they get frames laid out by the call graph and overlaid like those of any routine, and are only reachable through the dispatcher.

Events and interrupts. Calling an event from an interrupt handler is the intended use, and it needs no `Save`: the compiler wraps the call as `Save(0) ... Restore` itself, which preserves the working registers and the storage of the dispatcher and every handler (and what they call). Writing your own `Save` around it is allowed; then no second one is added. Keep two things in mind:

- A handler that runs from an interrupt gets the same respect as the interrupt itself. It interrupts the main program at any point, so keep it short, and do not make it call routines the main program also uses unless you save them (those calls still need `Save`, `E-ISR-CALL`).
- A **high and a low priority interrupt must never call the same event handler**, because one could interrupt the other halfway through it.

Example: a module that raises an event from an interrupt. `MyRX.bas` owns the interrupt and the receive hardware; the program decides what a received byte means. The handler gets the byte as an argument.

```
Module MyRX

Public Type TRxEvent = Event(ByVal c As Byte)
Dim FOnData As TRxEvent

Interrupt OnRX()
    FOnData(RC1REG)            ' the module reads the byte and passes it on
End Interrupt

Public Sub Initialize(ByVal pOnData As TRxEvent)
    FOnData = pOnData
    ' ... set up the receiver and its PIEx bit here ...
    Enable(OnRX)
End Sub
```

```
Program MyRXProgram
Device = 18F27Q10
Clock = 64
Config FEXTOSC = OFF, RSTOSC = HFINTOSC_64MHZ, WDTE = OFF, LVP = ON
Include "myrx"

Event GotChar(ByVal c As Byte)
    If c = 32 Then
        High(PORTB.0)
    ElseIf c = 42 Then
        High(PORTB.1)
    EndIf
End Event

MyRX.Initialize(GotChar)
While True
Wend
```

The program reacts to every received byte without containing or knowing about the interrupt, and the module can be reused with a different handler (or none: an unassigned `FOnData` does nothing).

Events and the library. The library modules never declare an `Interrupt` themselves, because a vector can be claimed only once and a handler without a priority would clash with a program that uses two priorities. The `timer` and `serial` modules instead provide an event variable and a `...Service` routine: assign your handler, then call the service routine from the main loop, or from your own interrupt (`Save(0, Timer0Service)` around the call). See `Timer0OnExpire` and `SerialOnRx` in section 3.

```
Include "timer"
Dim ticks As Byte
Event Tick()
    Inc(ticks)
End Event
Timer0OnExpire = Tick
Timer0Start(15, 255)
While True
    Timer0Service()          ' calls Tick once per Timer0 overflow
Wend
```

Differences from Swordfish. Swordfish gives every event its own frame that is never shared; here handler frames overlay by the call graph, which saves RAM, and the compiler's implicit `Save` (above) is what makes the call from an interrupt safe. Event parameters, comparison and arrays are additions.

Modules and Include

```
Module Name
    Public Sub Hello()
    End Sub
    ' statements here are start-up code, run before the main program
```

`Include "name"` loads `name.bas` from the including file's folder, then the project `lib/` folder, then any `-L` folder, once per compile. Only `Public` names are visible. Call as `Name.Routine()` or by the bare routine name when unambiguous. The spelling of a module's own name is what you write after the `.`: `Include "serial"` then `Serial.SerialInit(115200)`.

Device, Clock, Config

- **`Device = 18F27Q10`**: required (`E-NODEVICE`).
- **`Clock = n`** (or `Declare Xtal = n`): the instruction-clock source frequency in MHz that the program is built for, fractions allowed (`Clock = 0.5`). Delays, baud rates and timer periods are computed from it.
- **`Clock` does not set the hardware.** The oscillator is set by `Config RSTOSC` and by register writes. Either start the part at the speed you declare (`RSTOSC = HFINTOSC_64MHZ` with `Clock = 64`), or add `CSWEN = ON` and `Include "intosc"`, which writes `OSCFRQ`/`OSCCON1` for you. A mismatch with nothing setting the oscillator gives `W-CLOCK`. Details: `docs/CLOCK.md`.
- **`Config NAME = VALUE, ...`**: configuration words by pack names, validated against the device (`E-CONFIG`). Settings not mentioned take the pack default and the compiler prints a note listing them. Typical line:

```
Config FEXTOSC = OFF, RSTOSC = HFINTOSC_64MHZ, WDTE = OFF, LVP = ON
```

Preprocessor

Directive	Notes
`#define NAME [value]`, `#undefine NAME`	scoped to the file that defines them (only `-D` names are shared)
`#ifdef`, `#ifndef`, `#else`, `#endif`	
`#if`, `#elseif`, `#else`, `#endif`	expressions with numbers, `_clock`, `_device`, `_has(REG)` (true when the device description defines that register, or with `REG.FIELD` that bit field, any letter case: `#if _has(ANSELB)`, `#if _has(OSCFRQ.HFFRQ)`), `= <> < > <= >=`, `in (a, b, ...)`, `not`, `and`, `or`, parentheses. A device name is written bare: `#if _device = 18F27Q10`
`#error "text"`, `#warning "text"`	
`#option Optimize = Size|Speed|Balanced`, `Strict = true|false`, `ISR_SHADOW`, `Warn.<id> = true|false`	

Not supported: `#constant`, `#variable` (`E-PP-UNSUPPORTED`). Of the predefined names, only `_clock` and `_device` exist in `#if`. Directives inside `(* *)` or `{ }` comments are ignored.

2. Language commands

Entries are alphabetical. In the syntax lines `[x]` is optional and `{x}` is repeated. Every command is written with parentheses where it takes arguments (`High(led)`, not `High led`). Keywords are case-insensitive. The examples are fragments that assume the program header

```
Program Demo
Device = 18F27Q10
Clock = 16
Config FEXTOSC = OFF, RSTOSC = HFINTOSC_64MHZ, CSWEN = ON, WDTE = OFF, LVP = ON
Include "intosc"
```

followed by the lines shown. `Dim` lines come first, as the language requires.

Abs

Syntax

```
result = Abs(value)
```

Overview Absolute value of a number.

Parameters `value` is any numeric expression: a signed or unsigned integer, or a `Float`. The result has the type of `value`. An unsigned value is returned unchanged.

Example 1 An integer variable.

```
Dim a As Integer
Dim b As Integer
a = -1234
b = Abs(a)                ' 1234
```

Example 2 A `Float`: the sign bit is cleared.

```
Dim f As Float
Dim g As Float
f = -2.5
g = Abs(f)                ' 2.5
```

Notes A constant argument is folded at compile time. `Abs(-32768)` of an `Integer` (and the most negative value of any signed type) overflows and stays negative, as in any two's-complement system. The argument is evaluated more than once, so it must not contain a function call (a back-end error otherwise); assign the call to a variable first.

See also `Min`, `Max`.

AddressOf (and `@`)

Syntax

```
p = AddressOf(item)
p = @item
```

Overview The address of a variable, register, array, array element, structure field, constant array or routine.

Parameters `item` is the thing whose address is wanted. For RAM objects and registers the result is a `Word` holding the data-memory address (an array element with a run-time index gives the address of that element). For a `Const` array or a routine the result is a `LongWord` holding the 24-bit program-memory address (for a `Const` array, of its first byte). Anything else is `E-ADDR`.

Example 1 The address of a variable and of an array element.

```
Dim x As Byte
Dim a(4) As Byte
Dim k As Byte
Dim px As Word
Dim pe As Word
k = 2
px = AddressOf(x)
pe = @a(k)
```

Example 2 A routine and a constant table, as program-memory addresses.

```
Const Tbl(3) As Byte = (1, 2, 3)
Dim pr As LongWord
Dim pt As LongWord
Sub Blink()
    Inc(PORTB)
End Sub
pr = AddressOf(Blink)
pt = @Tbl
```

Notes The address of a routine keeps it in the program even if nothing calls it. The address of a bit, a `Const` array element chosen at run time, or an expression is not supported. An index in the operand must not contain a call. The address of a `ByRef` parameter is the address of the caller's variable.

See also `Asm`, `Bound`.

Asm ... End Asm

Syntax

```
Asm [Preserves(W, STATUS, BSR)]
    instruction ...
End Asm
```

Overview Raw `pic-as` source, passed through unchanged. The compiler assumes the block changes `WREG`, `STATUS` and `BSR` unless `Preserves(...)` says otherwise, and will not move its own code across it. A variable used only inside `Asm` still gets storage. Variables are referred to by their emitted symbol names (see `docs/ABI.md`; the names are in the `.S` and `.map` files).

Parameters `Preserves(...)` is optional and lists the registers the block leaves unchanged.

Example

```
Asm Preserves(W, STATUS, BSR)
    nop
    nop
End Asm
```

Notes Comments inside the block are assembler comments (`;`). The block ends at a line holding `End Asm`. Stack use of `call` in raw `Asm` is counted by the `E-STACK` check.

Bound

Syntax

```
n = Bound(array)
```

Overview Highest valid index of an array. For a declared array it is a constant (`Dim a(5)` gives 4, because an array of 5 has indexes 0..4). For an array **parameter** it is a run-time `Word`.

Example

```
Dim a(5) As Byte
Dim i As Byte
For i = 0 To Bound(a)
    a(i) = i
Next
```

See also `For ... Next`, `Clear`.

Break

Syntax

```
Break
```

Overview Leaves the innermost `For`, `While`, `Repeat` or `Do` loop. Outside a loop it is an error (`E-LOOP`).

Example

```
Dim x As Byte
While True
    Inc(x)
    If x > 3 Then
        Break
    EndIf
Wend
```

See also `Continue`, `Exit`.

Clear

Syntax

```
Clear(variable)
```

Overview Sets a variable, array, string or structure to zero.

Example

```
Dim a(4) As Byte
Dim w As Word
Clear(a)
Clear(w)
```

Continue

Syntax

```
Continue
```

Overview Jumps to the next iteration of the innermost loop (`E-LOOP` outside a loop). In a `For` loop the variable is stepped first.

Example

```
Dim i As Byte
Dim n As Byte
For i = 0 To 9
    If i = 5 Then
        Continue
    EndIf
    Inc(n)
Next
```

See also `Break`.

Dec

Syntax

```
Dec(variable [, amount])
```

Overview Subtract one from an integer variable, array element or structure field, or subtract `amount`. Not for `Float`.

Parameters `variable` is the item to decrement. `amount` is an optional integer constant, variable or expression; it is converted to the type of `variable`.

Example 1

```
Dim n As Byte
n = 10
Repeat
    Dec(n)
Until n = 0
```

Example 2 Subtracting an amount.

```
Dim level As Word
Dim amt As Byte
level = 1000
amt = 25
Dec(level, amt)          ' 975
Dec(level, 75)            ' 900
```

Notes On an array element with a run-time index the index is evaluated twice, so it must not contain a call. The result wraps like any assignment. A bit cannot take an amount. **See also** `Inc`.

DelayMS / DelayUS

Syntax

```
DelayMS(milliseconds)
DelayUS(microseconds)
```

Overview Cycle-counted delays computed from `Clock`. The compiler works out the loop counts at compile time. With a constant argument the achieved error is reported.

Parameters A constant, or a variable of any integer type. A negative constant is an error. A **variable** delay needs an integral number of instruction cycles per unit at the given `Clock`; otherwise the build fails rather than rounding silently.

Example

```
Dim led As PORTB.0
ANSELB = 0
Output(led)
While True
    Toggle(led)
    DelayMS(500)
Wend
```

Notes Interrupts lengthen a delay. The simulator suite checks exact cycle counts (for example 1,600, 16,000 and 8,000,000 cycles for the test delays). `DelaySeconds` is in the `delay` module.

See also `Clock` (section 1), `delay` module.

Disable / Enable

Syntax

```
Enable(HandlerName)
Disable(HandlerName)
```

Overview `Enable` assigns the handler to its vector and sets the global interrupt bits (`GIE/GIEH`, `PEIE/GIEL`); `Disable` clears them. They do not enable a particular source: set its `PIEx` bit yourself. Enabling two handlers of the same priority is an error (`E-ENABLE`).

Example

```
Dim ticks As Byte
Sub Count()
    Inc(ticks)
End Sub
Interrupt Tick()
    Save(0, Count)
        Count()
    Restore
    ' clear the flag of your interrupt source here
End Interrupt
' set up the source and its PIEx enable bit first (the timer module has
TimerNEnableInterrupt)
Enable(Tick)
Disable(Tick)
```

See also *Interrupts, events and context saving* (section 1), `Save`.

Do ... Loop

Syntax

```
Do
    ...
Loop

Do While condition ... Loop
Do Until condition ... Loop
Do ... Loop While condition
Do ... Loop Until condition
```

Overview Accepted as spellings of `While`/`Repeat`: `Do...Loop` = `While True`; `Do While c` = `While c`; `Do Until c` = `While Not c`; `Loop While c` = `Repeat ... Until Not c`; `Loop Until c` = `Repeat ... Until c`.

Example

```
Dim x As Byte
Do
    Inc(x)
Loop Until x > 3
Do While x > 0
    Dec(x)
Loop
```

See also `While`, `Repeat`.

Exit / ExitProc

Syntax

```
Exit
ExitProc                    ' alternative spelling
```

Overview Leaves the current `Sub`, `Function` or `Proc`. For a `Function` the current `Result` is returned.

Example

```
Function Clamp(ByVal a As Byte) As Byte
    Result = 100
    If a >= 100 Then
        Exit
    EndIf
    Result = a
End Function
Dim x As Byte
x = Clamp(42)
```

See also `Break`.

For ... Next ... Step

Syntax

```
For variable = start To end [Step constant]
    ...
Next
```

Overview `variable` must be an integer variable (not `Float`, not a bit). `end` is evaluated once. `Step` must be a constant; a negative `Step` counts down. After a loop that ran, the variable holds the last value it took. The variable must not be changed inside the loop (`E-LOOPVAR`). A `Byte` loop to 255 is safe (it does not wrap forever).

Example 1

```
Dim i As Byte
Dim a(8) As Byte
For i = 0 To Bound(a)
    a(i) = i * 2
Next
```

Example 2 Counting down by two.

```
Dim i As Byte
For i = 10 To 0 Step -2
    DelayMS(1)
Next
```

See also `Break`, `Continue`, `Bound`.

GoTo

Syntax

```
GoTo label
If condition GoTo label
```

Overview Jump to a label in the same routine. A label is a name followed by a colon at the start of a line. `GoSub`/`Return`, `Call` and computed `On GoTo` are not supported.

Example

```
Dim x As Byte
Again:
Inc(x)
If x < 5 GoTo Again
```

Notes `GoTo` and labels are not allowed inside a `Save` block.

High / Low / Toggle / Input / Output

Syntax

```
High(bit)         ' drive 1
Low(bit)          ' drive 0
Toggle(bit)       ' invert
Input(bit)        ' make an input (TRIS = 1)
Output(bit)       ' make an output (TRIS = 0)
```

Overview `bit` is a `Bit`/`Boolean` variable or alias, or a register bit such as `PORTB.4`. `High`, `Low` and `Toggle` act on the **latch** (`LATx`), and `Input`/`Output` on `TRISx`, so a pin set high is not disturbed by read-modify-write of other pins (`High(led)` on `PORTB.0` generates `bsf LATB,0`). To set a whole port direction or value, write the register: `TRISB = 0`, `LATB = $FF`.

Example

```
Dim led As PORTB.0
GpioAllDigital()          ' needs Include "gpio"; or write ANSELB = 0
Output(led)
High(led)
Toggle(led)
If led = 1 Then           ' a bit alias is compared, not used bare
    Low(led)
EndIf
```

Notes Parentheses are required: `High LED` and `High 1` (pin numbers) are **not** accepted. Pins reset to analog, so make them digital first. Pin numbers for the library routines are 0..23 (`RA0` = 0, `RB0` = 8, `RC0` = 16).

See also `gpio` module.

If ... Then ... ElseIf ... Else ... EndIf

Syntax

```
If condition Then
    ...
ElseIf condition Then
    ...
Else
    ...
EndIf

If condition GoTo label
```

Overview The condition must be Boolean (`E-COND`); `And` and `Or` short-circuit. Bit aliases are compared: `If led = 1 Then`.

Example

```
Dim x As Byte
x = 7
If x < 5 Then
    x = 0
ElseIf x < 10 Then
    x = 1
Else
    x = 2
EndIf
```

Notes The single-line form `If x Then y = 2` is rejected with a message; use a block or `If c GoTo label`.

Inc

Syntax

```
Inc(variable [, amount])
```

Overview Add one to an integer variable, array element or structure field, or add `amount`. Not for `Float`.

Parameters `variable` is the item to increment. `amount` is an optional integer constant, variable or expression; it is converted to the type of `variable`.

Example 1

```
Dim n As Byte
Repeat
    Inc(n)
Until n > 10
```

The loop above is the equivalent of `For n = 1 To 10`.

Example 2 Adding an amount.

```
Dim w As Word
Dim amt As Byte
w = 1
amt = 10
Repeat
    Inc(w, amt)
Until w > 100
```

Notes On an array element with a run-time index the index is evaluated twice, so it must not contain a call. The result wraps like any assignment. A bit cannot take an amount.

See also `Dec`.

Min / Max

Syntax

```
r = Min(a, b)
r = Max(a, b)
```

Overview Smaller / larger of two integers (`E-TYPE` for `Float`). The result has the common type of the two arguments, and mixed signedness compares in that common type.

Example 1

```
Dim a As Word
Dim hi As Word
a = 500
hi = Max(a, 1000)          ' 1000
```

Example 2 Limiting a value to a range.

```
Dim v As Integer
v = -40
v = Max(v, -10)           ' lower limit
v = Min(v, 10)            ' upper limit
```

Notes Constant arguments are folded at compile time. Both arguments are evaluated more than once, so they must not contain a function call; assign the call to a variable first.

See also `Abs`.

Repeat ... Until

Syntax

```
Repeat
    ...
Until condition
```

Overview The body runs at least once; the loop ends when the condition is true.

Example

```
Dim x As Byte
Repeat
    Inc(x)
Until x = 3
```

See also `While`, `Do`.

Save ... Restore

Syntax

```
Save(item {, item})
    ...
Restore
```

Overview Preserves the named variables, routines' storage (including what those routines call) and, with `0`, the compiler's working registers around the block. Required around any call to a routine made from an interrupt handler (otherwise `E-ISR-CALL`); a call through an event variable adds its own.

Parameters Each `item` is a variable, a routine, or `0`.

Example

```
Dim count As Byte
Sub Bump()
    Inc(count)
End Sub
Interrupt Tick()
    Save(0, Bump)
        Bump()
    Restore
End Interrupt
```

Notes `Exit`, `Break` and `Continue` inside a `Save` block restore first; `GoTo` and labels inside it are not supported.

See also *Interrupts, events and context saving* (section 1).

Select … Case … EndSelect

Syntax

```
Select expression
    Case 1
        ...
    Case 2, 3
        ...
    Case 10 To 20
        ...
    Case < 0
        ...
    Case Else          ' or: Else
        ...
EndSelect
```

Overview `Select` takes an integer expression. Case values are constants; a case can be a list, a range (`lo To hi`) or a relation (`< v`, `> v`, ...). Overlapping cases are an error (`E-CASE-OVERLAP`); an `Else` that can never run is the warning `W-CASE-ELSE`.

Example

```
Dim x As Byte
x = 7
Select x
    Case 1
        x = 10
    Case 2, 3
        x = 20
    Case 5 To 9
        x = 30
    Case > 100
        x = 40
    Case Else
        x = 0
EndSelect
```

Notes The form `Select Case x` is **not** accepted: write `Select x`. Strings and `Float`s cannot be selected on.

Terminate

Syntax

```
Terminate()
```

Overview Stops the program: it falls into the halt loop (a self-branch).

Example

```
Dim x As Byte
x = 1
Terminate()
```

While ... Wend

Syntax

```
While condition
    ...
Wend
```

Overview The condition is tested first. `While True ... Wend` is the usual main loop.

Example

```
Dim led As PORTB.0
ANSELB = 0
Output(led)
While True
    Toggle(led)
    DelayMS(250)
Wend
```

See also `Repeat`, `Do`, `Break`.

3. Library modules

Modules are in `lib/` and loaded with `Include "name"`. Call as `Module.Routine(...)` or by the bare routine name when unambiguous. **A routine is only known after its module is included**: without `Include "gpio"`, `GpioAllDigital()` is `E-UNDECLARED`. Only the routines a program calls are linked. All modules cite the PIC18F27Q10 data sheet (DS40002043F) and errata (DS80000832G) for their register values.

Each entry below has a **Syntax**, an **Overview**, **Parameters**, what it **Returns** (functions), and a short **Example**. The examples are fragments: they assume the program header

```
Program Demo
Device = 18F27Q10
Clock = 16
Config FEXTOSC = OFF, RSTOSC = HFINTOSC_64MHZ, CSWEN = ON, WDTE = OFF, LVP = ON
Include "intosc"
```

followed by the `Include` named in the module heading, and then the lines shown. `Dim` lines come first, as the language requires.

Common rules

- **Pins** are given by number: `RA0..RA7` = 0..7, `RB0..RB7` = 8..15, `RC0..RC7` = 16..23.
- **Return codes**: functions that can fail return a `Byte`: 1 = success, 0 = refused (bad pin, bad argument, conflict), unless the entry says otherwise. Bus modules also keep a status code.
- The reset state of port pins is **analog** (`ANSELx = 1`). Call `GpioAllDigital()` (after `Include "gpio"`) or write `ANSELx` before using pins digitally.
- **No module is reentrant.** Do not call a module routine from an interrupt handler and from the main program, or from two handlers, without `Save`.
- Library timing is computed from the compile-time `Clock`. If you change the oscillator at run time (`SetClock`) the library timing is then wrong (by design).
- **Not yet checked on hardware** for any module.

gpio (`Include "gpio"`)

Port-level helpers for ports A, B and C. Single pins need no library: `Input`, `Output`, `High`, `Low`, `Toggle` are language commands and `Dim led As PORTB.0` names a pin.

GpioAllDigital

Syntax

```
GpioAllDigital()
```

Overview Makes every pin of ports A, B and C a digital pin (`ANSELA/B/C = 0`). `ANSELx` resets to 1 (analog, input buffer off), so a pin read as digital stays 0 until this runs [DS40002043F p.247].

Example

```
Dim led As PORTB.0
GpioAllDigital()
Output(led)
High(led)
```

Notes Call it before `AdcPin` if the same program also uses analog inputs, then re-select those pins with `AdcPin`.

GpioPullUps

Syntax

```
GpioPullUps(maskA, maskB, maskC)
```

Overview Writes `WPUA/B/C`. A 1 bit enables the weak pull-up of that pin [p.251].

Parameters `maskA`, `maskB`, `maskC` are `Byte` constants or variables, bit n = pin Rx*n*.

Example

```
GpioAllDigital()
GpioPullUps(0, 0, %00000001)      ' pull-up on RC0
```

Notes The mask replaces the whole register: pins not set in the mask lose their pull-up.

GpioOpenDrain

Syntax

```
GpioOpenDrain(maskA, maskB, maskC)
```

Overview Writes `ODCONA/B/C`. A 1 bit makes the pin drive low only (open drain) [p.256].

Parameters As `GpioPullUps`.

Example

```
GpioOpenDrain(0, %00000010, 0)    ' RB1 open drain
```

delay (`Include "delay"`)

`DelayMS` and `DelayUS` are language commands (section 2). The module adds:

DelaySeconds

Syntax

```
DelaySeconds(s)
```

Overview Waits `s` seconds, as `s` calls of `DelayMS(1000)`.

Parameters `s` is a `Byte` (0..255).

Example

```
DelaySeconds(5)
```

Notes Interrupts lengthen the delay.

DelayNop

Syntax

```
DelayNop()
```

Overview Waits exactly one instruction cycle.

Example

```
Dim p As PORTB.0
Output(p)
High(p) : DelayNop() : Low(p)
```

intosc (`Include "intosc"`, internal oscillator set-up)

`Clock` only tells the compiler what speed to expect; it does not touch the hardware. Including `intosc` makes the hardware match it. Its start-up code (run before the main program) writes `OSCFRQ` and `OSCCON1` and waits, each wait bounded, for `OSCCON3.ORDY` and `OSCSTAT.HFOR`. Include it **first**: module start-up code runs in include order. For `Clock` of 1 MHz or more the divider is never raised, so start-up writes `OSCFRQ` then `OSCCON1` without reading `OSCCON2`; below 1 MHz it reads `OSCCON2` to pick the order.

Accepted `Clock` values (anything else is a compile error): 64, 48, 32, 16, 12, 8, 4, 2, 1 MHz, and 0.5, 0.25, 0.125, 0.0625, 0.03125 MHz (1 MHz divided by 2 to 32). `Config` needs `CSWEN = ON` unless `Clock` already equals the `RSTOSC` speed. If a write is ignored, `SysClockOk()` is False.

```
Program Blink
Device = 18F27Q10
Clock = 16
Config FEXTOSC = OFF, RSTOSC = HFINTOSC_64MHZ, CSWEN = ON, WDTE = OFF, LVP = ON
Include "intosc"
```

SetClock

Syntax

```
ok = SetClock(mhz)
```

Overview Runs HFINTOSC at `mhz` with no divider, at run time.

Parameters `mhz` is a `Byte`: 1, 2, 4, 8, 12, 16, 32, 48 or 64.

Returns `Boolean`: True when the switch completed and read back; False (nothing written) for another value, or when the clock did not switch.

Example

```
Dim ok As Boolean
ok = SetClock(4)
```

Notes `DelayMS`, `DelayUS` and the serial, I2C, SPI and PWM modules keep timing for the compile-time `Clock`, so they are wrong until the clock is back at `Clock`.

SetClockDiv

Syntax

```
ok = SetClockDiv(mhz, shift)
```

Overview As `SetClock`, with the clock divided by 2^`shift`.

Parameters `mhz` as `SetClock`; `shift` is 0..9 (1:1 to 1:512).

Returns `Boolean`, as `SetClock`.

Example

```
Dim ok As Boolean
ok = SetClockDiv(16, 2)           ' 16 MHz / 4 = 4 MHz
```

system (`Include "system"`, reset cause and clock check)

SysResetCause

Syntax

```
cause = SysResetCause()
```

Overview Reports the cause of the latest Reset whose `PCON0` flags are not yet cleared.

Returns `Byte`: 1 power-on, 2 brown-out, 3 MCLR pin, 4 watchdog time-out, 5 watchdog window violation, 6 `RESET` instruction, 7 stack overflow, 8 stack underflow, 0 none. Tested in the order 1, 2, 7, 8, 5, 4, 3, 6.

Example

```
Dim c As Byte
c = SysResetCause()
SysClearResetFlags()
```

Notes Hardware never clears the flags, so read the cause at start and then call `SysClearResetFlags`; otherwise an old cause is reported again.

SysResetFlags

Syntax

```
raw = SysResetFlags()
```

Overview Returns the raw `PCON0` value.

Returns `Byte`.

Example

```
Dim f As Byte
f = SysResetFlags()
```

SysClearResetFlags

Syntax

```
SysClearResetFlags()
```

Overview Puts every `PCON0` flag to its inactive state so the next Reset shows only its own cause.

Example

```
SysClearResetFlags()
```

SysClockHz

Syntax

```
hz = SysClockHz()
```

Overview The system clock in Hz as the oscillator is set now (from `OSCCON2` and `OSCFRQ`).

Returns `LongWord`; 0 when the source is not HFINTOSC (external and low-frequency sources have no frequency the module knows). The nominal HFINTOSC frequency is used; `OSCTUNE` is not applied.

Example

```
Dim hz As LongWord
hz = SysClockHz()
```

SysClockFrom

Syntax

```
hz = SysClockFrom(cosc, cdiv, hffrq)
```

Overview The same conversion from given field values.

Parameters `cosc` is `OSCCON2` bits 6:4 (6 = HFINTOSC); `cdiv` is `OSCCON2` bits 3:0 (divider code 0..9); `hffrq` is `OSCFRQ` bits 3:0 (codes 0..8 = 1, 2, 4, 8, 12, 16, 32, 48, 64 MHz).

Returns `LongWord`; 0 for any other source or a reserved code.

Example

```
Dim hz As LongWord
hz = SysClockFrom(6, 0, 5)        ' HFINTOSC, 1:1, 16 MHz
```

SysClockOk

Syntax

```
ok = SysClockOk()
```

Overview True when the oscillator runs at the frequency the program was compiled for (`Clock`).

Returns `Boolean`.

Example

```
If SysClockOk() = False Then
    Terminate()
EndIf
```

timer (`Include "timer"`, Timer0, Timer1, Timer2)

The module configures a timer and reads its flag. It declares no interrupt handler: to be interrupted, call `TimerNEnableInterrupt`, write your own `Interrupt`, and call `TimerNAck` in it (flags are not cleared by hardware). All timers count FOSC/4 unless a Timer1 source says otherwise. `prescale` ranges are per timer. External pins (T1CKI, T1G, T2IN) must be digital inputs; the module does not set them up.

Timer0Start

Syntax

```
Timer0Start(prescale, period)
```

Overview Starts Timer0 as an 8-bit periodic timer counting 0..`period`. One overflow takes `(period + 1) * 2^prescale` instruction cycles.

Parameters `prescale` 0..15 (1:1 to 1:32768); `period` 0..255.

Example

```
Timer0Start(7, 124)               ' 16 MHz: 125 * 128 cycles = 4 ms
```

Timer0Start16

Syntax

```
Timer0Start16(prescale)
```

Overview Starts Timer0 as a 16-bit free-running timer, read with `Timer0Read`.

Parameters `prescale` 0..15.

Example

```
Timer0Start16(4)
```

Timer0Read

Syntax

```
count = Timer0Read()
```

Overview Reads the 16-bit count (`TMR0L` first, which latches `TMR0H`).

Returns `Word`.

Example

```
Dim t As Word
Timer0Start16(4)
t = Timer0Read()
```

Timer0Expired

Syntax

```
flag = Timer0Expired()
```

Overview True once per overflow; clears the flag when it reports it.

Returns `Boolean`.

Example

```
Dim led As PORTB.0
ANSELB = 0
Output(led)
Timer0Start(15, 244)
While True
    If Timer0Expired() Then
        Toggle(led)
    EndIf
Wend
```

Timer0Ack

Syntax

```
Timer0Ack()
```

Overview Clears the Timer0 flag. Call it from your own interrupt handler, wrapped in `Save` (a call from a handler is otherwise `E-ISR-CALL`).

Example

```
Interrupt Tick()
    Save(0, Timer0Ack)
        Timer0Ack()
    Restore
End Interrupt
Timer0Start(15, 244)
Timer0EnableInterrupt()
Enable(Tick)
```

Timer0EnableInterrupt / Timer0DisableInterrupt

Syntax

```
Timer0EnableInterrupt()
Timer0DisableInterrupt()
```

Overview Set or clear `PIE0.TMR0IE`. Global interrupts are enabled separately with `Enable`.

Example

```
Timer0EnableInterrupt()
Timer0DisableInterrupt()
```

Timer0OnExpire / Timer0Service

Syntax

```
Timer0OnExpire = Handler          ' Type TTimerEvent = Event()
Timer0Service()
```

Overview `Timer0OnExpire` is an event variable (type `TTimerEvent`, declared by the module). `Timer0Service` checks the Timer0 flag; if the timer has overflowed it clears the flag and calls the handler once. `Timer1OnExpire`/`Timer1Service` and `Timer2OnExpire`/`Timer2Service` do the same for Timer1 and Timer2. With no handler assigned nothing is called, but the flag is still cleared. Call the service routine from the main loop, or from your own interrupt (wrap it in `Save(0, Timer0Service)`). The module declares no `Interrupt`, so it never claims a vector.

Example

```
Dim ticks As Byte
Event Tick()
    Inc(ticks)
End Event
Timer0OnExpire = Tick
Timer0Start(15, 255)
While ticks < 10
    Timer0Service()
Wend
```

See also *Events* (section 1), `Timer0Expired`, `Timer0EnableInterrupt`.

Timer0Stop

Syntax

```
Timer0Stop()
```

Overview Stops Timer0.

Example

```
Timer0Stop()
```

Timer1Start

Syntax

```
Timer1Start(prescale)
```

Overview Starts Timer1 as a 16-bit timer on FOSC/4, ungated. It overflows every `65536 * 2^prescale` cycles.

Parameters `prescale` 0..3 (1:1 to 1:8).

Example

```
Timer1Start(3)                    ' 16 MHz / 4 / 8 = 500 kHz
While Timer1Expired() = False
Wend
```

Timer1StartSource

Syntax

```
Timer1StartSource(clk, prescale)
```

Overview Timer1 from any clock source, free-running and ungated. External sources are synchronised to the system clock (`SYNC = 0`), so pulses must be longer than a system clock cycle; asynchronous operation is not offered.

Parameters `clk` is one of `T1ClkPin` (T1CKI pin, rising edges, default RC0), `T1ClkFosc4`, `T1ClkFosc`, `T1ClkHfIntOsc`, `T1ClkLfIntOsc`, `T1ClkMfIntOsc`, `T1ClkSosc`, `T1ClkTmr0`. `prescale` 0..3.

Example

```
Timer1StartSource(T1ClkFosc, 0)
```

Notes `T1ClkSosc` needs `OSCEN.SOSCEN` set and time to stabilise. In counter mode a falling edge must be seen before the first rising edge counts, so the first pulse after a start may be missed (data sheet 20.2.2).

Timer1StartCounter

Syntax

```
Timer1StartCounter(prescale)
```

Overview Counts rising edges on the T1CKI pin; the same as `Timer1StartSource(T1ClkPin, prescale)`.

Example

```
Dim n As Word
Dim t1cki As PORTC.0
GpioAllDigital()
Input(t1cki)
Timer1StartCounter(0)
n = Timer1Read()
```

Timer1StartGated

Syntax

```
Timer1StartGated(clk, prescale, source, activeHigh)
```

Overview Counts only while the gate source is at the given level. The gate is set up before the timer is switched on, so no count is taken ungated.

Parameters `clk` and `prescale` as `Timer1StartSource`. `source` is `T1GatePin` (T1G, RB5), `T1GateTmr0`, `T1GateTmr2`, `T1GateCcp1`, `T1GateCcp2`, `T1GateCmp1` or `T1GateCmp2`. `activeHigh` is True to count while the gate is high, False while low.

Example

```
Timer1StartGated(T1ClkFosc4, 0, T1GatePin, True)
```

Timer1StartPulse

Syntax

```
Timer1StartPulse(clk, prescale, source, activeHigh, fullCycle)
```

Overview Times the first pulse of the gate source after the call, then holds. Poll `Timer1PulseDone`, then read `Timer1Read` for the length in clock counts.

Parameters As `Timer1StartGated`, plus `fullCycle`: True toggles the gate on every rising edge, so the measured time is a whole period of the source instead of its high (or low) time.

Example

```
Dim w As Word
Timer1StartPulse(T1ClkFosc4, 0, T1GatePin, True, False)
While Timer1PulseDone() = False
Wend
w = Timer1Read()
```

Timer1PulseDone

Syntax

```
flag = Timer1PulseDone()
```

Overview True once the pulse started by `Timer1StartPulse` has ended.

Returns `Boolean`.

Example

```
While Timer1PulseDone() = False
Wend
```

Timer1GateState

Syntax

```
level = Timer1GateState()
```

Overview The live state of the gate (`T1GCON.GVAL`); it is not affected by the gate enable.

Returns `Boolean`.

Example

```
Dim g As Boolean
g = Timer1GateState()
```

Timer1Read / Timer1Expired / Timer1Ack / Timer1Stop

Syntax

```
count = Timer1Read()
flag  = Timer1Expired()
Timer1Ack()
Timer1Stop()
```

Overview As the Timer0 routines, on `PIR4.TMR1IF`. `Timer1Read` returns a `Word` (with `RD16` set, reading `TMR1L` latches `TMR1H`).

Example

```
Dim t As Word
Timer1Start(0)
t = Timer1Read()
Timer1Stop()
```

Timer1EnableInterrupt / Timer1DisableInterrupt

Syntax

```
Timer1EnableInterrupt()
Timer1DisableInterrupt()
```

Overview Set or clear `PIE4.TMR1IE`.

Example

```
Timer1EnableInterrupt()
```

Timer2Start

Syntax

```
Timer2Start(prescale, postscale, period)
```

Overview Starts Timer2 as a free-running 8-bit timer on FOSC/4, counting 0..`period`. The flag is set every `(period + 1) * 2^prescale * (postscale + 1)` cycles.

Parameters `prescale` 0..7 (1:1 to 1:128); `postscale` 0..15 (1:1 to 1:16); `period` 0..255.

Example

```
Timer2Start(7, 0, 249)            ' 16 MHz: 250 * 128 cycles = 8 ms
```

Notes `T2TMR` is not cleared by a `T2CON` write. The `pwm` module owns Timer2 while a channel runs.

Timer2StartLimit

Syntax

```
Timer2StartLimit(prescale, postscale, period, mode, source)
```

Overview Timer2 with a hardware-limit mode. Writing `ON` arms the timer: in the edge and level modes it waits for the reset source; in the one-shot modes it runs once and clears `ON`.

Parameters `prescale`, `postscale`, `period` as `Timer2Start`. `mode` is one of `T2ModeSoftware`, `T2ModeGateHigh`, `T2ModeGateLow`, `T2ModeResetAnyEdge`, `T2ModeResetRising`, `T2ModeResetFalling`, `T2ModeResetLow`, `T2ModeResetHigh`, `T2ModeOneShot`, `T2ModeOneShotRising`, `T2ModeOneShotFalling`, `T2ModeOneShotAnyEdge`, `T2ModeOneShotRisingReset`, `T2ModeOneShotFallingReset`, `T2ModeOneShotRisingLow`, `T2ModeOneShotFallingHigh`, `T2ModeMonoRising`, `T2ModeMonoFalling`, `T2ModeMonoAnyEdge`, `T2ModeOneShotLevelHigh`, `T2ModeOneShotLevelLow`. `source` is `T2RstPin` (T2IN, default RC3), `T2RstTmr4`, `T2RstTmr6`, `T2RstCcp1`, `T2RstCcp2`, `T2RstPwm3`, `T2RstPwm4`, `T2RstCmp1`, `T2RstCmp2`.

Example

```
Timer2StartLimit(7, 0, 249, T2ModeOneShot, T2RstPin)
While Timer2Running()
Wend
```

Notes Edge-triggered modes need six timer clocks between triggers, level modes a level of at least three (data sheet 21.5).

Timer2Running

Syntax

```
flag = Timer2Running()
```

Overview True while the timer is on (`T2CON.ON`). Use this, not the flag, to detect the end of a one-shot: the data sheet does not say `TMR2IF` is set when `ON` clears.

Returns `Boolean`.

Example

```
Dim r As Boolean
r = Timer2Running()
```

Timer2Read / Timer2Expired / Timer2Ack / Timer2Stop

Syntax

```
count = Timer2Read()
flag  = Timer2Expired()
Timer2Ack()
Timer2Stop()
```

Overview As the Timer0 routines, on `PIR4.TMR2IF`. `Timer2Read` returns a `Byte`. Stopping the timer also resets its counters.

Example

```
Dim t As Byte
Timer2Start(0, 0, 255)
t = Timer2Read()
Timer2Stop()
```

Timer2EnableInterrupt / Timer2DisableInterrupt

Syntax

```
Timer2EnableInterrupt()
Timer2DisableInterrupt()
```

Overview Set or clear `PIE4.TMR2IE`.

Example

```
Timer2EnableInterrupt()
```

Hardware untested for external clock, gate and pulse use.

serial (`Include "serial"`, EUSART1, 8N1, TX = RC6, RX = RC7)

`Clock` must be FOSC in MHz. The internal oscillator drifts with temperature and voltage, so use a crystal for anything beyond casual use. There is no formatted `Print`: build output from the routines below (for a signed number or a `Float`, convert in your own code).

SerialInit

Syntax

```
SerialInit(baud)
```

Overview Routes EUSART1 through PPS (TX on RC6, RX on RC7), sets the pin directions and `BRG16 = BRGH = 1`, computes the baud divisor from `Clock` (one 32-bit divide at run time, rounded to the nearest value) and enables TX and RX.

Parameters `baud` is a `LongWord`, for example 9600 or 115200.

Example

```
SerialInit(115200)
```

SerialWriteByte

Syntax

```
SerialWriteByte(b)
```

Overview Sends one byte, waiting until the transmit register is free.

Example

```
SerialWriteByte(65)               ' "A"
```

SerialWrite

Syntax

```
SerialWrite(text)
```

Overview Sends a string or literal, up to 80 characters.

Example

```
SerialWrite("hello")
```

SerialNewLine

Syntax

```
SerialNewLine()
```

Overview Sends CR + LF.

Example

```
SerialWrite("done")
SerialNewLine()
```

SerialWriteWord

Syntax

```
SerialWriteWord(v)
```

Overview Sends a `Word` as unsigned decimal, without leading zeros.

Example

```
SerialWriteWord(1234)
```

SerialWriteHex

Syntax

```
SerialWriteHex(v)
```

Overview Sends a `Byte` as two hexadecimal digits.

Example

```
SerialWriteHex($A5)               ' "A5"
```

SerialAvailable

Syntax

```
flag = SerialAvailable()
```

Overview True when a received byte is waiting.

Returns `Boolean`.

Example

```
Dim b As Byte
If SerialAvailable() Then
    b = SerialRead()
EndIf
```

SerialRead

Syntax

```
b = SerialRead()
```

Overview Waits for a byte and returns it. An overrun stops the receiver until `CREN` is cycled; this routine cycles it.

Returns `Byte`.

Example

```
Dim b As Byte
b = SerialRead()
SerialWriteByte(b)                ' echo
```

SerialOnRx / SerialRxService

Syntax

```
SerialOnRx = Handler              ' Type TRxEvent = Event(ByVal c As Byte)
SerialRxService()
```

Overview `SerialOnRx` is an event variable whose handler receives each byte. `SerialRxService` clears an overrun (as `SerialRead` does) and, if a byte has arrived, reads it (which clears `RCIF`) and calls the handler with it. With no handler assigned the byte is read and dropped. Call it from the main loop, or from your own interrupt wrapped in `Save(0, SerialRxService)`.

Example

```
Dim last As Byte
Event OnByte(ByVal c As Byte)
    last = c
End Event
SerialInit(115200)
SerialOnRx = OnByte
While True
    SerialRxService()
Wend
```

See also *Events* (section 1), `SerialAvailable`, `SerialRead`.

SerialFlush

Syntax

```
SerialFlush()
```

Overview Waits until the last byte has left the shift register (`TRMT`).

Example

```
SerialWrite("bye")
SerialFlush()
```

serial2 (`Include "serial2"`, EUSART2, TX = RB6, RX = RB7)

The same nine routines with the prefix `Serial2`: `Serial2Init`, `Serial2WriteByte`, `Serial2Write`, `Serial2NewLine`, `Serial2WriteWord`, `Serial2WriteHex`, `Serial2Available`, `Serial2Read`, `Serial2Flush`. Syntax, parameters and returns are as in `serial`.

Example

```
Serial2Init(9600)
Serial2Write("hi")
Serial2NewLine()
```

Notes RB6/RB7 are the ICSP pins: disconnect the programmer while using them. Both serial modules can be used together. **Set-up verified only**: the simulator does not set `TX2IF`, so a real transmit is unproven.

adc (`Include "adc"`, ADCC, 10-bit, right-justified)

Software-started conversions with the clock derived from FOSC (`Clock` must be FOSC in MHz). Channel numbers 0..23 are pins (`RA0..RA7` = 0..7, `RB0..RB7` = 8..15, `RC0..RC7` = 16..23). Internal channels: `AdcChVss` 60, `AdcChTemp` 61, `AdcChDac` 62, `AdcChFvr` 63.

Errata handling: (1.4.4) the first conversion after a channel change is run and dropped; code that writes `ADPCH` itself bypasses this. (1.4.1) `AdcRefFvr` needs TAD of at least 8 us, which `ADCS` allows only up to `Clock` = 16 MHz; above that use the VDD reference.

AdcInit

Syntax

```
AdcInit()
```

Overview Turns the ADC on with VDD/VSS references and the FOSC clock chosen so TAD is at least 1 us (and at most 9 us). Hardware acquisition is set to 5 us, which suits a source impedance up to 10 kohm.

Example

```
AdcInit()
```

Notes A higher source impedance needs a longer time: write `ADACQ` after `AdcInit`.

AdcPin

Syntax

```
AdcPin(ch)
```

Overview Makes the pin of channel `ch` an analog input (`TRISx = 1`, `ANSELx = 1`).

Parameters `ch` 0..23. Other channel numbers are internal and need no pin.

Example

```
AdcPin(0)                       ' RA0
```

AdcRead

Syntax

```
raw = AdcRead(ch)
```

Overview Converts channel `ch` once.

Parameters `ch` 0..63.

Returns `Word`, 0..1023.

Example

```
Dim v As Word
AdcInit()
AdcPin(0)
v = AdcRead(0)
```

AdcReadAvg

Syntax

```
raw = AdcReadAvg(ch, count)
```

Overview Mean of `count` conversions, rounded down.

Parameters `ch` as `AdcRead`; `count` 1..64.

Returns `Word`; 0 for a `count` of 0.

Example

```
Dim v As Word
AdcInit()
AdcPin(1)
v = AdcReadAvg(1, 16)
```

AdcMillivolts

Syntax

```
mv = AdcMillivolts(raw, vrefMv)
```

Overview Converts a result to millivolts: `raw * vrefMv / 1023`.

Parameters `raw` a `Word` from `AdcRead`; `vrefMv` the reference in millivolts.

Returns `Word`.

Example

```
Dim v As Word
Dim mv As Word
AdcInit()
AdcPin(0)
v = AdcRead(0)
mv = AdcMillivolts(v, 3300)
```

AdcRefVdd

Syntax

```
AdcRefVdd()
```

Overview Positive reference = VDD, negative = VSS (the reset state).

Example

```
AdcRefVdd()
```

AdcRefFvr

Syntax

```
AdcRefFvr(gain)
```

Overview Positive reference = the fixed voltage reference; waits for `FVRRDY`.

Parameters `gain` 1, 2 or 4 (1.024, 2.048 or 4.096 V; the output cannot exceed VDD). 4 or more is taken as 4, 0 as 1.

Example

```
AdcRefFvr(2)                    ' 2.048 V
```

Notes Slows the conversion clock to TAD of at least 8 us (errata 1.4.1); not possible above `Clock` = 16 MHz. The temperature indicator needs `FVRCON.TSEN`, which the module does not set.

eeprom (`Include "eeprom"`, data EEPROM, 1024 bytes)

Global interrupts are held off for the unlock sequence and their previous state is restored. Success is judged by read-back (errata 1.6.1), not `NVMERR`. Reading stored values is **not verifiable in the simulator** (it does not model DFM reads); test on hardware.

EepromSize

Syntax

```
EepromSize
```

Overview Constant, 1024: the number of bytes.

Example

```
Dim last As Word
last = EepromSize - 1
```

EepromRead

Syntax

```
b = EepromRead(addr)
```

Overview Reads one byte.

Parameters `addr` is a `Word`, 0..1023.

Returns `Byte`; 255 for an address above 1023.

Example

```
Dim b As Byte
b = EepromRead(0)
```

EepromWrite

Syntax

```
ok = EepromWrite(addr, value)
```

Overview Skips the write if the byte already holds `value` (saves a cycle); otherwise erases and programs it with the unlock sequence and waits.

Parameters `addr` 0..1023; `value` a `Byte`.

Returns `Byte`: 1 when the byte reads back as `value`, 0 for a bad address or a failed write.

Example

```
Dim ok As Byte
ok = EepromWrite(0, 42)
```

EepromReadWord / EepromWriteWord

Syntax

```
w  = EepromReadWord(addr)
ok = EepromWriteWord(addr, value)
```

Overview 16-bit forms, low byte first at `addr` and `addr + 1`.

Parameters `addr` 0..1022; `value` a `Word`.

Returns `EepromReadWord` a `Word`; `EepromWriteWord` a `Byte`, 1 when both bytes were stored.

Example

```
Dim w As Word
Dim ok As Byte
ok = EepromWriteWord(10, 1234)
w = EepromReadWord(10)
```

pwm (`Include "pwm"`, CCP1, CCP2, PWM3, PWM4 on Timer2)

Four outputs on one timebase: channel 1 = CCP1, 2 = CCP2, 3 = PWM3, 4 = PWM4. All channels share the Timer2 period (the frequency); each has its own duty cycle. The module owns Timer2 while a channel runs, so do not use `Timer2` routines at the same time. CCP1/CCP2 can output on ports B and C (pins 8..23); PWM3/PWM4 on ports A and C. The duty is a number of steps out of `PwmSteps()`.

PwmInit

Syntax

```
ok = PwmInit(hz)
```

Overview Chooses the Timer2 prescale (1..128) and period for `hz`. Timer2 is started by `PwmStart`.

Parameters `hz` is a `LongWord`, 1..1000000.

Returns `Byte`: 1, or 0 when no prescale reaches `hz` or a channel is running (the period is shared).

Example

```
Dim ok As Byte
ok = PwmInit(1000)
```

PwmFrequency

Syntax

```
hz = PwmFrequency()
```

Overview The achieved frequency.

Returns `LongWord`; 0 before a successful `PwmInit`.

Example

```
Dim f As LongWord
PwmInit(1000)
f = PwmFrequency()
```

PwmSteps

Syntax

```
n = PwmSteps()
```

Overview Number of duty steps in one period: `4 * (T2PR + 1)`, at most 1024. The 10-bit resolution is reached only when `T2PR` = 255; a higher frequency gives fewer steps.

Returns `Word`.

Example

```
Dim n As Word
PwmInit(1000)
n = PwmSteps()
```

PwmStart

Syntax

```
ok = PwmStart(ch, pin, duty)
```

Overview Starts a channel on a pin, following the data sheet set-up order (driver off, routing, duty, Timer2 on, wait for its first overflow, then driver on) so the first period is complete.

Parameters `ch` 1..4; `pin` 0..23 (see the heading for valid pins); `duty` in steps, 0.. `PwmSteps()`.

Returns `Byte`: 0 (and nothing done) without a successful `PwmInit`, for a channel or pin that cannot be used, or for a pin another running channel uses. A running channel is restarted.

Example

```
Dim ok As Byte
PwmInit(1000)
ok = PwmStart(1, 18, 512)         ' CCP1 on RC2, half duty at 1024 steps
```

PwmDuty

Syntax

```
PwmDuty(ch, duty)
```

Overview Sets the duty in steps. No effect on a stopped channel.

Parameters `ch` 1..4; `duty` 0.. `PwmSteps()` (at most 1023 is representable).

Example

```
PwmDuty(1, 256)
```

Notes Duty registers are double buffered and copied at the period match; if the match falls between the two byte writes, one period can use a mixed value.

PwmPercent

Syntax

```
PwmPercent(ch, pct)
```

Overview Sets the duty as a percentage, rounded down to a step.

Parameters `ch` 1..4; `pct` 0..100. 100 gives the highest representable duty, which with 1024 steps is 1023/1024: the output cannot reach a true 100 %.

Example

```
PwmPercent(1, 25)
```

PwmInvert

Syntax

```
ok = PwmInvert(ch, inverted)
```

Overview Sets the output polarity of PWM3 or PWM4 (`PWMxCON.POL`); on a running channel it applies at once.

Parameters `ch` 3 or 4; `inverted` a `Boolean` .

Returns `Byte` : 0 for channels 1 and 2, which have no polarity bit.

Example

```
Dim ok As Byte
ok = PwmInvert(3, True)
```

PwmStop

Syntax

```
PwmStop(ch)
```

Overview Stops a channel: the pin becomes an input again and returns to its `LATxy` bit and the module is disabled. Timer2 is stopped when the last channel stops.

Parameters `ch` 1..4.

Example

```
PwmStop(1)
```

i2c (`Include "i2c"`, MSSP1 host)

I2C host (master). SCL and SDA go to any pin of ports B and C (8..23); the reset selection is SCL = RC3 (19) and SDA = RC4 (20). The module is the only host on the bus (no multi-master): a collision is an error. The bus needs external pull-ups. `Clock` must be FOSC in MHz. Every step is done by the hardware one at a time and each wait is bounded, so a stuck bus returns an error instead of hanging.

Status codes (`I2cStatus`, and the return of Start, Restart, Stop and Send): 0 ok, 1 timeout (a bus line held low or no pull-up), 2 bus collision, 3 no acknowledge, 4 module not initialised.

The module polls and clears `SSP1IF` itself; keep the MSSP1 interrupt disabled. It owns MSSP1: do not use `spi` at the same time.

I2cInit

Syntax

```
ok = I2cInit(hz, scl, sda)
```

Overview Sets up MSSP1 as an I2C host at about `hz`. The clock is `FOSC / (4 * (SSPxADD + 1))`; the divider is rounded up so the clock is at or below `hz`. The highest clock is FOSC/16 and the lowest FOSC/1024. Slew rate control is disabled for 100 kHz or less and for 1 MHz and above, enabled in between.

Parameters `hz` a `LongWord`, 1..1000000; `scl` and `sda` pin numbers 8..23, different.

Returns `Byte`: 1 when set up, 0 when a pin is not allowed or no divider of 3..255 reaches `hz`.

Example

```
Dim ok As Byte
ok = I2cInit(100000, 19, 20)
```

I2cOff

Syntax

```
I2cOff(scl, sda)
```

Overview Disables the MSSP and gives both pins back to their `LATxy` bits.

Parameters The same pins given to `I2cInit`.

Example

```
I2cOff(19, 20)
```

I2cStatus

Syntax

```
s = I2cStatus()
```

Overview The status of the last Start, Restart, Stop, byte or register routine.

Returns `Byte`, the codes above.

Example

```
Dim s As Byte
s = I2cStatus()
```

I2cStart / I2cRestart / I2cStop

Syntax

```
s = I2cStart()
s = I2cRestart()
s = I2cStop()
```

Overview Generate a start, repeated-start or stop condition.

Returns `Byte`, the status code.

Example

```
Dim s As Byte
s = I2cStart()
s = I2cSend($A0)                  ' address 0x50 + write
s = I2cStop()
```

I2cSend

Syntax

```
s = I2cSend(b)
```

Overview Sends one byte (an address with its R/W bit, or data).

Parameters `b` a `Byte`.

Returns `Byte`: 0 when the client acknowledged, 3 when it did not.

Example

```
Dim s As Byte
s = I2cStart()
s = I2cSend($A0)
s = I2cSend(0)
s = I2cStop()
```

I2cReceive

Syntax

```
b = I2cReceive(ack)
```

Overview Receives a byte, then sends an acknowledge or not-acknowledge.

Parameters `ack` True when more bytes follow, False for the last byte.

Returns `Byte`. On an error it returns 0 and `I2cStatus` is not 0.

Example

```
Dim b As Byte
Dim s As Byte
s = I2cStart()
s = I2cSend($A1)                  ' address 0x50 + read
b = I2cReceive(False)
s = I2cStop()
```

I2cPing

Syntax

```
found = I2cPing(addr)
```

Overview Tests whether a client acknowledges its address for writing.

Parameters `addr` the 7-bit address, 0..127.

Returns `Boolean`.

Example

```
Dim here As Boolean
here = I2cPing($50)
```

I2cWriteReg

Syntax

```
s = I2cWriteReg(addr, reg, value)
```

Overview Start, address + W, `reg`, `value`, Stop.

Parameters `addr` 7-bit address; `reg` and `value` are `Byte`s.

Returns `Byte`: the first status code that was not 0 (the Stop is always attempted).

Example

```
Dim s As Byte
s = I2cWriteReg($68, $6B, 0)
```

I2cReadReg

Syntax

```
v = I2cReadReg(addr, reg)
```

Overview Start, address + W, `reg`, Restart, address + R, one byte with not-acknowledge, Stop.

Returns `Byte`. On an error it returns 0 and `I2cStatus` is not 0.

Example

```
Dim v As Byte
v = I2cReadReg($68, $75)
```

spi (`Include "spi"`, MSSP1 host)

SPI host (master). SCK, SDO and SDI go to pins of ports B and C (8..23); SDO can be 255 for a receive-only host. The chip select is not part of the module: drive it with a port bit. `Clock` must be FOSC in MHz. The module owns MSSP1 (do not use `i2c` together) and does not use its interrupt.

Modes: 0 = idle low, data changes on the falling edge; 1 = idle low, rising; 2 = idle high, falling; 3 = idle high, rising (CPOL/CPHA as `CKP` / `CKE`). **Status** (`SpiStatus`): 0 after the last transfer completed, 1 after a timeout, 4 when not initialised.

SpiInit

Syntax

```
ok = SpiInit(hz, mode, sck, sdo, sdi)
```

Overview Sets up MSSP1 as an SPI host. The clock is FOSC/4 for `hz` of FOSC/4 or more, otherwise `FOSC / (4 * (SSPxADD + 1))`, rounded so the clock is at or below `hz`. The lowest is FOSC/1024.

Parameters `hz` a `LongWord`, 1..16000000; `mode` 0..3; `sck`, `sdo`, `sdi` pins 8..23 (`sdo` may be 255).

Returns `Byte`: 1, or 0 for a bad pin, a pin used twice, a mode above 3, or a clock below FOSC/1024.

Example

```
Dim ok As Byte
ok = SpiInit(1000000, 0, 19, 20, 21)
```

Notes Use `SpiOff` before changing pins.

SpiOff

Syntax

```
SpiOff(sck, sdo)
```

Overview Disables the MSSP and gives the pins back to their `LATxy` bits.

Parameters The pins given to `SpiInit`.

Example

```
SpiOff(19, 20)
```

SpiStatus

Syntax

```
s = SpiStatus()
```

Returns `Byte`, the codes above.

Example

```
Dim s As Byte
s = SpiStatus()
```

SpiTransfer

Syntax

```
r = SpiTransfer(b)
```

Overview Sends one byte and returns the byte received at the same time. The wait is bounded.

Returns `Byte`; 0 on an error (see `SpiStatus`).

Example

```
Dim r As Byte
r = SpiTransfer($9F)
```

SpiWrite

Syntax

```
SpiWrite(b)
```

Overview Sends one byte and ignores the byte received.

Example

```
SpiWrite($06)
```

SpiRead

Syntax

```
r = SpiRead()
```

Overview Receives one byte (sends 255 on SDO).

Returns `Byte`.

Example

```
Dim r As Byte
r = SpiRead()
```

lcd (`Include "lcd"`, HD44780 character LCD, 4-bit)

Six port pins are used: RS, E and DB4..DB7 (pins 0..23). R/W is not used: tie it to ground. The busy flag is not read; every instruction is followed by a fixed wait longer than its worst case. The pins are set by read-modify-write of `LATA/B/C`, so do not use the module from a handler and from the main program together, and do not change those `LATx` registers elsewhere while a routine runs.

A 2-line display has line 1 at DDRAM address 00H..27H and line 2 at 40H..67H; a 1-line display uses 00H..4FH. Four-line displays are not described by the data sheet: address them with `LcdSetAddress`.

LcdInit

Syntax

```
ok = LcdInit(rs, en, d4, d5, d6, d7, lines)
```

Overview Makes the six pins digital outputs, runs the 4-bit initialisation by instruction, turns the display on with no cursor, clears it and selects increment, no shift. Takes about 55 ms.

Parameters `rs`, `en`, `d4`..`d7` pin numbers 0..23, all different; `lines` 1 or 2 (5 x 8 dot font).

Returns `Byte`: 1, or 0 when `lines` is not 1 or 2, a pin is above 23 or a pin is used twice.

Example

```
Dim ok As Byte
ok = LcdInit(16, 17, 18, 19, 20, 21, 2)   ' RS=RC0 E=RC1 D4..D7=RC2..RC5
```

LcdCommand

Syntax

```
LcdCommand(c)
```

Overview Sends any instruction byte (see the data sheet's instruction table). Waits 2 ms after Clear display and Return home, 50 us after the rest.

Example

```
LcdCommand($01)                  ' clear display
```

LcdClear / LcdHome

Syntax

```
LcdClear()
LcdHome()
```

Overview `LcdClear` clears the display and puts the cursor at line 1, column 0. `LcdHome` puts the cursor there and shifts the display back to its unshifted position.

Example

```
LcdClear()
LcdHome()
```

LcdDisplay

Syntax

```
LcdDisplay(on, cursor, blink)
```

Overview Display on/off control.

Parameters Three `Boolean`s: display on, cursor shown, character at the cursor blinks.

Example

```
LcdDisplay(True, True, False)
```

LcdShift

Syntax

```
LcdShift(whole, right)
```

Overview Shifts the whole display (`whole` True) or only the cursor, one position right (`right` True) or left.

Example

```
LcdShift(True, False)             ' whole display one place left
```

LcdGoto

Syntax

```
LcdGoto(col, row)
```

Overview Moves the cursor.

Parameters `col` 0..39 (0..79 on a one-line display); `row` 0 or 1. A position the display does not have is ignored.

Example

```
LcdGoto(0, 1)                     ' start of line 2
```

LcdSetAddress

Syntax

```
LcdSetAddress(addr)
```

Overview Sets the raw DDRAM address; the next character is written there.

Parameters `addr` 0..127.

Example

```
LcdSetAddress($14)                ' line 3 of a common four-line display
```

LcdWriteChar

Syntax

```
LcdWriteChar(ch)
```

Overview Writes one character code at the cursor. Codes 32..126 are ASCII (A00 character ROM); 0..7 are the user characters of `LcdCustomRow`.

Example

```
LcdWriteChar(65)                  ' "A"
```

LcdWrite

Syntax

```
LcdWrite(text)
```

Overview Writes a string or literal (up to 80 characters) at the cursor.

Example

```
LcdWrite("Hello")
```

LcdWriteWord

Syntax

```
LcdWriteWord(v)
```

Overview Writes a `Word` as unsigned decimal, without leading zeros.

Example

```
LcdWriteWord(2048)
```

LcdWriteHex

Syntax

```
LcdWriteHex(v)
```

Overview Writes a `Byte` as two hexadecimal digits.

Example

```
LcdWriteHex($3C)
```

LcdCustomRow

Syntax

```
LcdCustomRow(slot, row, bits)
```

Overview Sets one dot row of a user-defined character.

Parameters `slot` 0..7 (shown as character code 0..7); `row` 0..7; `bits` uses the low five bits (bit 4 = leftmost dot). Keep row 7 at 0 if the cursor is used.

Example

```
LcdCustomRow(0, 0, %00001110)
LcdCustomRow(0, 1, %00010001)
LcdGoto(0, 0)                    ' leave CGRAM before writing text
LcdWriteChar(0)
```

Notes The write leaves the address counter in CGRAM: call `LcdGoto`, `LcdClear` or `LcdHome` before writing text.

oled (`Include "oled"`, SSD1309, 128 x 64, I2C or 4-wire SPI)

Set up the bus first: `I2cInit` at no more than 400 kHz, or `SpiInit` in mode 0 at no more than 10 MHz. The module owns no pins except D/C#, CS# and RES# (pins 0..23; 255 for CS# or RES# means not controlled). The panel supply (about 12 V) is external: switch it on after RES#. There is **no frame buffer** (the serial interfaces are write-only): you write pages of 8 pixel rows, one byte per column, bit 0 on top, with `OledGoto` and `OledData`. Text uses a 5 x 7 font in 6 x 8 cells (21 characters on each of 8 lines) for ASCII 32..126. There is no pixel, line or bitmap drawing. If a module shows every other line or an upside-down picture, use `OledFlip`.

OledInitI2c

Syntax

```
ok = OledInitI2c(addr, res)
```

Overview Starts the display on I2C, runs the start-up command sequence, clears the RAM and switches the display on.

Parameters `addr` the 7-bit address, 60 or 61 (0x3C/0x3D); `res` the RES# pin, or 255.

Returns `Byte`: 1 when every byte was acknowledged, else 0 (`OledStatus` gives the I2C code).

Example

```
Dim ok As Byte
ok = I2cInit(400000, 19, 20)
ok = OledInitI2c(60, 255)
```

OledInitSpi

Syntax

```
ok = OledInitSpi(dc, cs, res)
```

Overview Starts the display on 4-wire SPI.

Parameters `dc` the D/C# pin; `cs` the CS# pin or 255; `res` the RES# pin or 255.

Returns `Byte`: 1, or 0 for a pin above 23 or a pin used twice.

Example

```
Dim ok As Byte
ok = SpiInit(4000000, 0, 19, 20, 255)
ok = OledInitSpi(16, 17, 18)
```

OledStatus / OledClearStatus

Syntax

```
s = OledStatus()
OledClearStatus()
```

Overview `OledStatus` is 0 when all I2C bytes since the init (or the last `OledClearStatus`) were acknowledged, else the first I2C status code (3 = no acknowledge). Always 0 for SPI.

Returns `Byte`.

Example

```
Dim s As Byte
s = OledStatus()
OledClearStatus()
```

OledOn

Syntax

```
OledOn(isOn)
```

Overview Display on, or off (sleep). The RAM keeps its content.

Example

```
OledOn(False)
```

OledContrast

Syntax

```
OledContrast(level)
```

Overview Sets the contrast; reset value is 127.

Parameters `level` 0..255.

Example

```
OledContrast(200)
```

OledInvert

Syntax

```
OledInvert(inverse)
```

Overview Inverse display (True) or normal (False).

Example

```
OledInvert(True)
```

OledAllOn

Syntax

```
OledAllOn(allOn)
```

Overview Every pixel on regardless of the RAM (True), or the RAM content (False).

Example

```
OledAllOn(True)
```

OledFlip

Syntax

```
OledFlip(rotated)
```

Overview Turns the picture through 180 degrees (True) or normal (False). Takes effect for data written afterwards; the RAM is not rewritten.

Example

```
OledFlip(True)
OledClear()
```

OledGoto

Syntax

```
OledGoto(col, page)
```

Overview The next data byte goes to column `col` of page `page`. After each byte the column advances and wraps to the next page at the end of the line.

Parameters `col` 0..127; `page` 0..7.

Example

```
OledGoto(0, 0)
```

OledTextGoto

Syntax

```
OledTextGoto(x, line)
```

Overview The next character goes to cell `x` of text line `line`.

Parameters `x` 0..20; `line` 0..7. An `x` above 20 is ignored.

Example

```
OledTextGoto(0, 3)
```

OledData

Syntax

```
OledData(b)
```

Overview Writes one column byte (bit 0 on top) at the current position.

Example

```
OledGoto(10, 2)
OledData(%11111111)               ' a vertical bar 8 pixels high
```

OledFill / OledClear

Syntax

```
OledFill(pattern)
OledClear()
```

Overview `OledFill` writes `pattern` to the whole RAM and returns to column 0, page 0. `OledClear` is `OledFill(0)`.

Example

```
OledFill($AA)                     ' stripes
OledClear()
```

OledWriteChar

Syntax

```
OledWriteChar(ch)
```

Overview One character at the current position: five font columns and one blank. Codes outside 32..126 show `?`.

Example

```
OledTextGoto(0, 0)
OledWriteChar(65)
```

OledWrite

Syntax

```
OledWrite(text)
```

Overview Writes a string or literal (up to 80 characters).

Example

```
OledTextGoto(0, 0)
OledWrite("Hello")
```

OledWriteWord / OledWriteHex

Syntax

```
OledWriteWord(v)
OledWriteHex(v)
```

Overview A `Word` as unsigned decimal without leading zeros; a `Byte` as two hex digits.

Example

```
OledTextGoto(0, 1)
OledWriteWord(1234)
OledWriteHex($FF)
```

FontByte

Syntax

```
b = FontByte(idx)
```

Overview Byte `idx` of the font table: 5 column bytes for each code 32..126, bit 0 on top.

Parameters `idx` a `Word`, 0..474.

Returns `Byte`.

Example

```
Dim b As Byte
b = FontByte(0)
```

aht30 (`Include "aht30"`, AHT30 temperature and humidity sensor, I2C)

Set up the bus first: `I2cInit` at no more than 400 kHz, with pull-ups of 2.0 to 4.7 kOhm. The sensor is at 7-bit address 0x38. A reading is two steps: `Aht30Start` triggers a measurement, and `Aht30Fetch` reads it at least 80 ms later; `Aht30Measure` does both and waits. Sample at most once a second, or the sensor's self-heating passes 0.1 C. The temperature comes back as an `Integer` in 1/100 degree C (2512 = 25.12 C) and the humidity as a `Word` in 1/100 %RH (4560 = 45.60 %RH); they are written only when the status is 0.

Status codes 0 ok; 1 timeout; 2 bus collision; 3 no acknowledge; 4 `I2c` not initialised; 5 busy (`Aht30Fetch` was called before the measurement finished); 6 CRC mismatch.

Example

```
Dim t As Integer
Dim rh As Word
Dim st As Byte
st = I2cInit(100000, 19, 20)
Aht30Init()
st = Aht30Measure(t, rh)
If st = 0 Then
    ' t = 2512 and rh = 4560 mean 25.12 C and 45.60 %RH
End If
```

floatlib (automatic)

Not included by hand. Routines `FAdd FSub FMul FDiv`, `FEq FNe FLt FLe FGt FGe`, `FFromLong FFromULong FToLong` operate on `LongWord` bit patterns. You do not call them; the compiler does when a program uses `Float`.

4. Diagnostics

Every message has a code, `error[E-TYPE]` or `warning[W-NARROW]`. Full list with meanings: `docs/DIAGNOSTICS.md`. The ones met most often:

Code	Cause
`E-NODEVICE`, `E-DEVICE`	no `Device`, or no description for it
`E-SYNTAX`	the text does not match the grammar (for example a single-line `If`, `High 1`, `Select Case`)
`E-ORDER`	a `Dim`, `Include` or routine after the first statement
`E-UNDECLARED`	misspelled name, or a module not `Include`d
`E-CONFIG`	unknown `Config` name or value
`E-TYPE`, `E-SIGN`, `E-MIX`, `E-NARROW` / `W-NARROW`	type rules
`E-COND`	condition is not Boolean (a bare bit alias)
`E-CONST-RANGE`, `E-BOUNDS`	constant does not fit; constant index out of range
`E-RAM`, `E-STACK`	variables exceed RAM; hardware stack deeper than 31
`E-RECURSION`	call cycle
`E-ISR-CALL`	call from a handler without `Save ... Restore` (also float maths)
`E-BACKEND`	valid per the front end but not generated yet (see below)
`E-UNSUPPORTED`	reserved but not available (`BitOf`, `Declare` other than `Xtal`, ...)
`W-CLOCK`	`Clock` differs from the `RSTOSC` speed and nothing sets the oscillator
`W-REGNAME`	a declaration hides a register name

`E-BACKEND` ("not supported by the back end yet") means the front end accepted a rare form that the code generator does not handle: a `Reentrant` routine, a bit passed `ByRef`, a call inside the index of a bit, an array of strings, a function returning an array or String, and similar. The compiler reports these rather than miscompiling them.

Appendix A: Alternative spellings and unsupported forms

Accepted alternative spellings

`Declare Xtal = n`; `Symbol`; `Proc`/`EndProc`/`ExitProc`; `Do`/`Loop` forms; type names `SByte`, `SWord`, `Dword`, `Long` (and conversions); `$FF`/`0xFF`, `%1010`/`0b1010`; `:` between statements; `'` and `(* *)` comments; `_` continuation; `Dim x As Type`; `Toggle`, `Inc`, `Dec`, `DelayMS`, `DelayUS`, `High`, `Low`, `Input`, `Output` **with parentheses**; `Abs`; `Bound`.

Not available (or different)

Feature of other BASIC compilers	basil 0.1.9
Other devices	only PIC18F27Q10
Statements without parentheses (`High LED`, `DelayMs 500`, `HRsout "x"`)	parentheses required
`Declare` switches (LCD pins, baud, pin routing)	only `Declare Xtal`; peripherals take arguments in `...Init(...)` calls
`PinHigh`, `PinLow`, `PinSet`, `PinGet`, `PinMode`, pin numbers in `High`	use register bits / aliases
`Pin` type, `SDword`, `Double`, 24-bit `Long`, `Global`/`Shared` variables	not accepted (`Static` and `Volatile` are supported, see *Variables and types*)
`Print`, `Cls`, `Cursor`, `Plot`, `Line`, graphic LCD commands	`lcd` module routines; no graphics
`HRsout`, `HRsin`, `HSerout`, `Rsout`, `Serin`, `Serout`	`serial` / `serial2` routines
`Dec`, `Hex`, `Bin`, `Str$`, `Val` formatting modifiers	`SerialWriteWord`, `SerialWriteHex` only
`ADin`, `Pot`, `RCin`	`adc` module
`HPWM`, `PWM`, `Servo`, `Freqout`, `Sound`	`pwm` module (no servo/sound)
`Busin/Busout`, `I2Cin/I2Cout`, `Hbusin`, `Shin/Shout`, `Owrite/Oread`, `USBin`	`i2c`, `spi` modules; no 1-wire, USB, software SPI
`Read/Write EEPROM`, `Edata`, `Cdata`, `LookUp`, `LookDown`	`eeprom` module; `Const` arrays
`Branch`, `On GoTo`, `On GoSub`, `GoSub`, `Return`, `Call`	not available; use `Select` and routines
`Select Case x`	`Select x`
`Break`, `Continue`, `While/Wend`, `Repeat/Until`, `For/Next`	supported
`Sin`, `Cos`, `Sqr`, `Log`, `Exp`, `Pow`, `ISqr`, `Div32`, `Dcd`, `Ncd`, `@`, `?`	not available; `Abs`, `Min`, `Max` and arithmetic only
`Len`, `Left$`, `Mid$`, `Right$`, string arrays	not available
`Tern`, `SizeOf`	not available
`$config`, `Config` directive	`Config NAME = VALUE` with pack names

Feature of other BASIC compilers	basil 0.1.9
`$include`, `$asm`, `Asm`	`Include "name"`, `Asm ... End Asm` (pic-as syntax)
Interrupt via `On_Hardware_Interrupt`	`Interrupt Name() ... End Interrupt` plus `Enable`
`0x` and `0b` literals	accepted, as are `$` and `%`
Overloading, `Reentrant` routines	not yet

Appendix B: Test checklist

Work down this list. Each item names what to observe.

Build and size

1. `basil --version` prints `0.1.9`. `basil --check` on a program with a deliberate error prints a code, line, column and caret.
2. Compile `my_src/first_test/main.bas`; confirm `.hex`, `.S`, `.map` appear and the size line prints.
3. Compile the same program with `-O0`; it should behave the same and be larger.

Language (all verifiable on the bench with one LED and a scope or serial port) 4. Blink with `DelayMS(500)` at your `Clock`: measure the period (should be 1.000 s). Repeat at 64, 32, 16, 8, 4, 1 MHz. 5. `DelayUS` with a pin toggle: measure 10, 100, 1000 us. 6. Integer arithmetic at each width: results printed over serial for `+ - * / Mod << >>` against known values. 7. `Select` with list, range and relational cases; `For` with negative `Step`; `Break`/`Continue`. 8. Arrays: initialiser, `Bound`, `Const` array read, copy; string append and compare. 9. Structure and union field access; `ByRef` parameter modifying the caller's variable. 10. Float: `+ - * /`, comparison, conversion to integer, print the integer part. 11. Interrupt: Timer0 interrupt toggling a pin; then a second priority; a call from the handler wrapped in `Save`.

Library (hardware only; the simulator cannot show these) 12. `serial`: 9600 and 115200 baud at 16 MHz and 64 MHz; check the characters and the error at each. Echo using `SerialAvailable`/`SerialRead`. 13. `serial2` on RB6/RB7 with the programmer disconnected. 14. `adc`: a potentiometer on RA0, readings 0..1023; the internal FVR channel; `AdcRefFvr(1)` scale. 15. `pwm`: frequency and duty on a scope on CCP1/CCP2/PWM3/PWM4 pins; `PwmInvert`; 100 % corner case. 16. `i2c`: `I2cPing` against a known device; read a register; remove the pull-up and confirm status 1. 17. `spi`: loop MOSI to MISO and confirm `SpiTransfer` returns what was sent, in all four modes. 18. `eeprom`: write, power-cycle, read back; byte and word forms; an out-of-range address. 19. `timer`: Timer0/1/2 flag periods; Timer1 as an external counter on RC0; gate and pulse measurement. 20. `lcd` and `oled`: text, cursor, custom character, contrast, invert, flip. 21. `intosc`/`system`: `Clock` at several values; `SysClockHz()` and `SysClockOk()` agree with a scope; `SysResetCause()` after power-on, MCLR and a watchdog reset.

Record anything that disagrees with this manual as a bug against the entry it came from.